

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Giới thiệu về hệ sinh thái Hadoop

Pseudo-Distributed

Môn học: Nhập môn dữ liệu lớn

Sinh viên thực hiện:

Lê Quốc Anh (23127150)

Giáo viên hướng dẫn:

TS. Nguyễn Ngọc Thảo

ThS. Huỳnh Lâm Hải Đăng

CN. Trần Huy Bân

Ngày 12 tháng 7 năm 2026

Mục lục

1	Giới thiệu Hadoop MapReduce và nội dung phần Pseudo mode	1
2	Cài đặt Hadoop với Pseudo mode trên máy ảo Ubuntu (VMware Workstation)	1
2.1	Cài đặt máy ảo Ubuntu trên VMware Workstation	1
2.2	Cài đặt Hadoop với Pseudo-distributed mode trên máy ảo Ubuntu	3
2.2.1	Cài đặt Java phù hợp với phiên bản Hadoop	7
2.2.2	Cài đặt SSH	7
2.2.3	Tải và cài đặt Hadoop	9
2.2.4	Cấu hình Pseudo-distributed mode trên Hadoop	13
2.2.5	Khởi động HDFS	17
2.3	Thao tác với Hadoop	18
2.3.1	Task 1: Tạo một thư mục /hcmus trên HDFS	20
2.3.2	Task 2: Tạo một user với tên khtn_23127150	20
2.3.3	Task 3: Tạo subfolder /hcmus/23127150 trên HDFS và upload một file vào đó	22
2.3.4	Task 4: chmod 744 và đổi owner của /hcmus/23127150 thành khtn_23127150	23
2.4	Chạy file JAR verify	23
3	Một số lỗi xảy ra trong quá trình	27
3.1	HDFS không start thành công sau khi reboot máy	27
4	Warm up với WordCount	33
4.1	Cài đặt sbt trên Ubuntu	33
4.2	Tạo project Scala với sbt	34
4.3	Viết code Scala cho WordCount	36
4.3.1	WordCount.scala	36
4.3.2	Mapper.scala	38
4.3.3	Reducer.scala	39
4.3.4	Flow chạy của WordCount	40
4.4	Build JAR file và chạy với Hadoop	41
	Tài liệu	49

1 Giới thiệu Hadoop MapReduce và nội dung phần Pseudo mode

Apache Hadoop là một framework mã nguồn mở cho phép lưu trữ và xử lý dữ liệu lớn theo mô hình phân tán trên nhiều máy tính thông thường. Hai thành phần cốt lõi của Hadoop gồm HDFS (Hadoop Distributed File System) đảm nhận việc lưu trữ dữ liệu và MapReduce đảm nhận việc xử lý dữ liệu. Trong HDFS, mỗi file được chia thành các block và phân tán trên nhiều DataNode, còn NameNode giữ toàn bộ metadata (thông tin về thư mục, file, vị trí block). Nhờ cơ chế này, Hadoop có thể lưu trữ được lượng dữ liệu vượt xa dung lượng của một máy đơn lẻ và vẫn đảm bảo khả năng chịu lỗi.

MapReduce là mô hình lập trình xử lý dữ liệu theo hai pha chính. Pha *Map* đọc dữ liệu đầu vào và biến đổi thành các cặp key-value trung gian; sau đó Hadoop thực hiện *shuffle & sort* để gom tất cả các value có cùng key lại; cuối cùng pha *Reduce* tổng hợp mỗi nhóm key để cho ra kết quả. Nhờ chia nhỏ công việc và chạy song song trên nhiều node, MapReduce có thể xử lý khối lượng dữ liệu rất lớn một cách hiệu quả, đồng thời tự động phân phối lại công việc khi có node gặp sự cố.

Pseudo-distributed mode (gọi tắt là *pseudo mode*) là chế độ chạy Hadoop trên một máy duy nhất, nhưng mỗi daemon (NameNode, DataNode, ...) lại chạy trong một tiến trình Java riêng biệt, mô phỏng gần đúng cách hoạt động của một cluster thật. Đây là chế độ phù hợp để học tập và kiểm thử trước khi triển khai fully distributed mode trên nhiều máy.

Trong phần cá nhân này, em sẽ lần lượt trình bày các nội dung sau:

- Cài đặt máy ảo Ubuntu trên VMware Workstation.
- Cài đặt và cấu hình Hadoop 3.5.0 ở chế độ pseudo-distributed mode.
- Thực hiện các thao tác cơ bản với HDFS theo yêu cầu của Lab 1.
- Chạy file JAR verify để kiểm tra kết quả.
- Xử lý một số lỗi gặp phải trong quá trình cài đặt.
- Giải bài toán WordCount bằng Hadoop MapReduce viết bằng Scala.

2 Cài đặt Hadoop với Pseudo mode trên máy ảo Ubuntu (VMware Workstation)

2.1 Cài đặt máy ảo Ubuntu trên VMware Workstation

Để cài đặt Hadoop với Pseudo mode trên máy ảo Ubuntu, trước hết cần tải về những thứ sau:

- VMWare Workstation: Em sử dụng VMWare Workstation 17 Pro tải về từ trang chủ VMware Workstation [8].
- Ubuntu ISO: Em sử dụng Ubuntu 26.04 LTS (phiên bản mới nhất hiện tại) tải về từ trang tải Ubuntu Desktop [4].

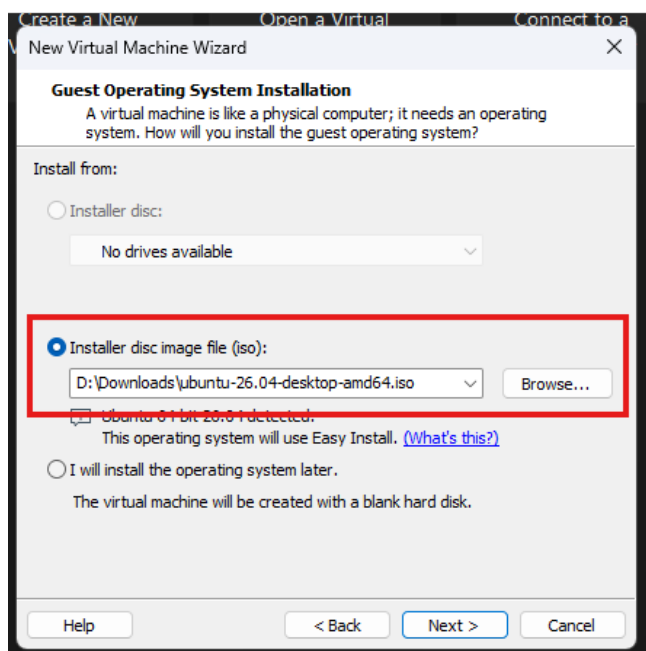
Sau đây là các bước để cài đặt một máy ảo Ubuntu trên VMware Workstation:

1. Mở VMWare Workstation và chọn “Create a New Virtual Machine”.
2. Giao diện cài đặt hiện lên, chọn “Typical (recommended)” và nhấn “Next” (Hình 1).



Hình 1: Chọn “Typical (recommended)” khi tạo máy ảo mới.

3. Chọn “Installer disc image file (iso)” và nhấn “Browse” để chọn file ISO của Ubuntu đã chuẩn bị trước đó (Hình 2).



Hình 2: Chọn file ISO của Ubuntu.

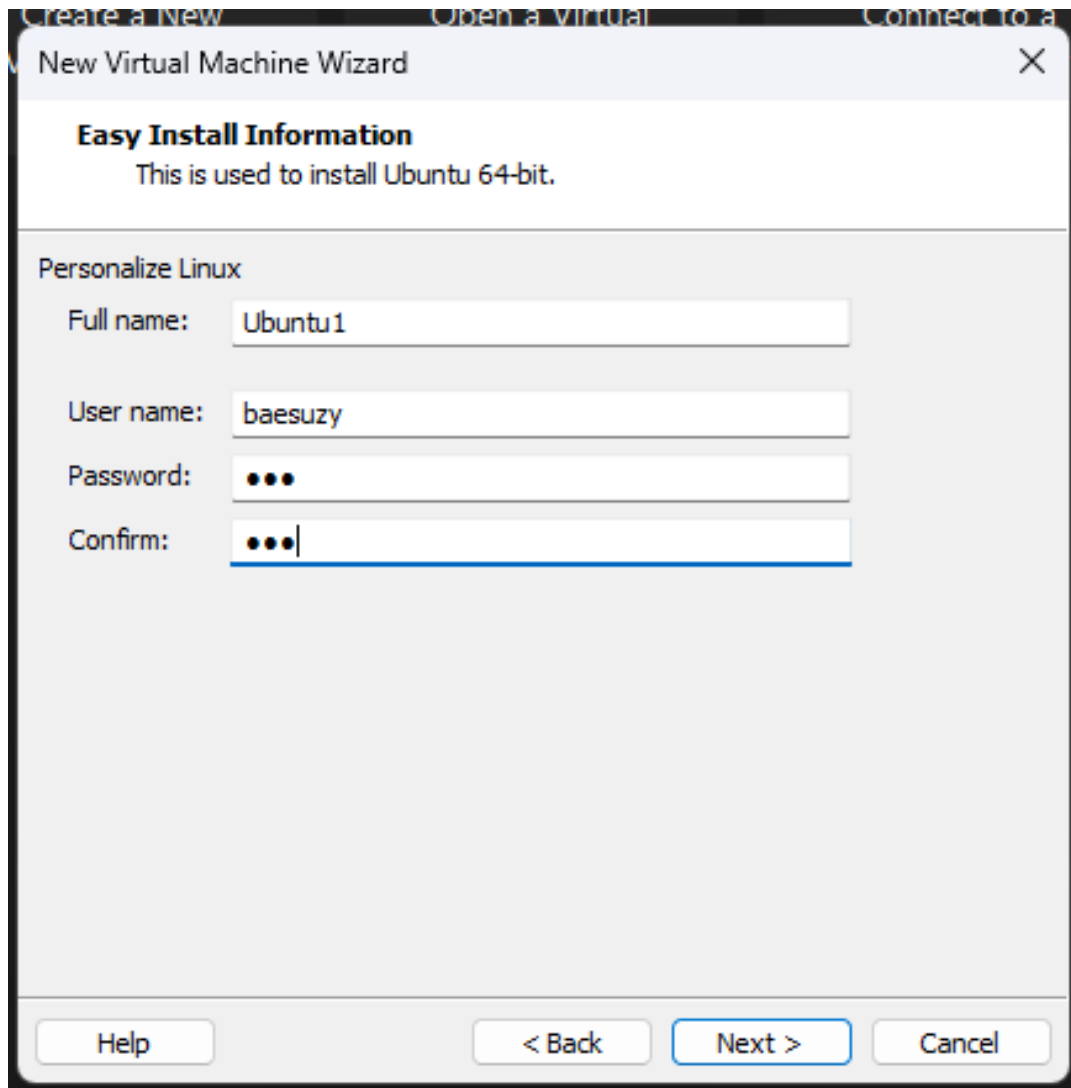
4. Điền các thông tin như yêu cầu (Hình 3).
5. Chọn vị trí lưu trữ máy ảo và nhấn “Next” (Hình 4).
6. Chọn dung lượng ổ cứng cho máy ảo và nhấn “Next”. Em chọn theo recommend là 20 GB (Hình 5).
7. Còn lại các bước em để mặc định và nhấn “Finish” để hoàn tất quá trình cài đặt máy ảo Ubuntu.

Sau đó thì giao diện cài đặt trên Ubuntu sẽ hiện ra và em tiến hành cài đặt Ubuntu như bình thường. Sau khi cài đặt xong, em tiến hành đăng nhập vào Ubuntu và mở terminal để cài đặt Hadoop.

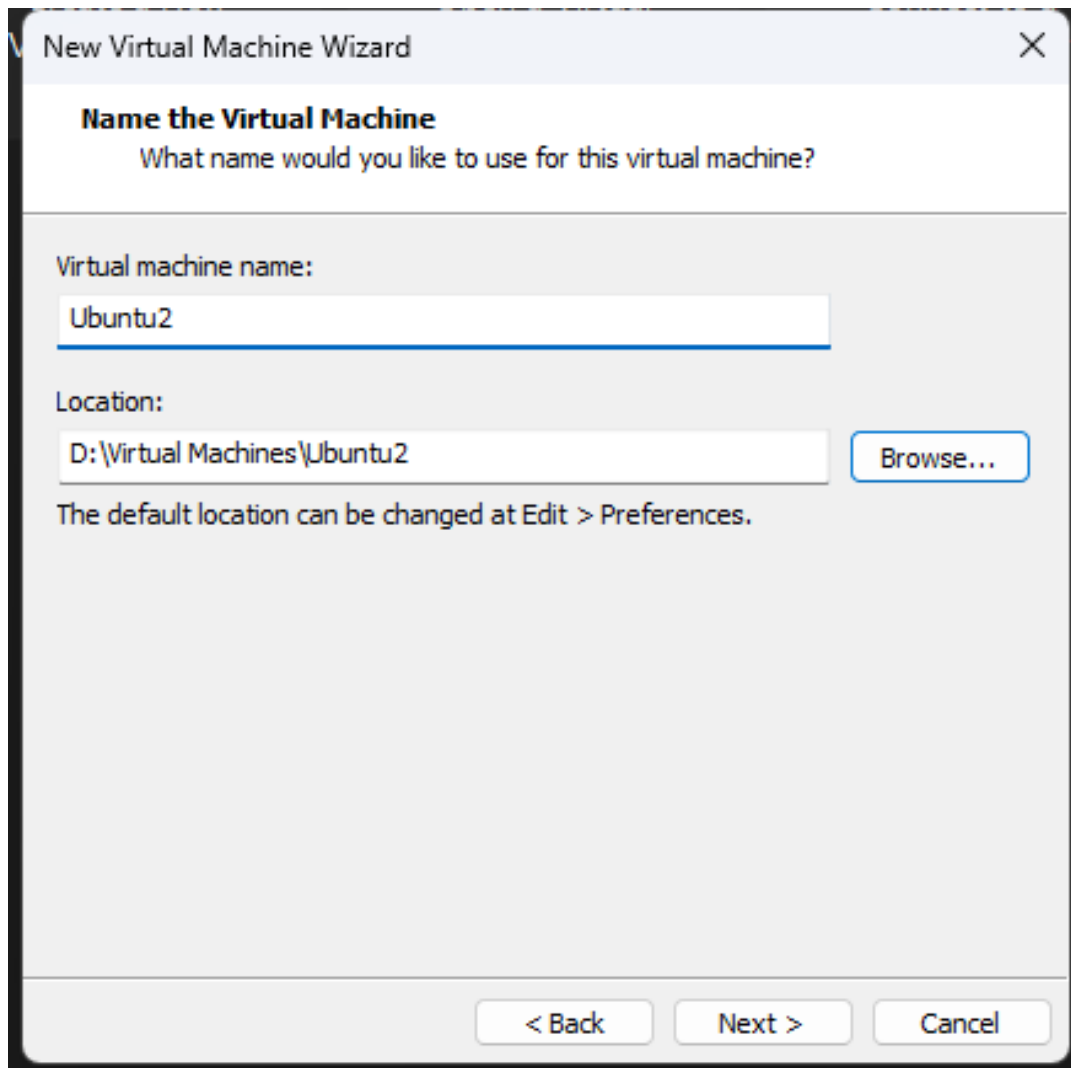
2.2 Cài đặt Hadoop với Pseudo-distributed mode trên máy ảo Ubuntu

Phần cài đặt này em tham khảo theo hướng dẫn Single Cluster của Hadoop [1].

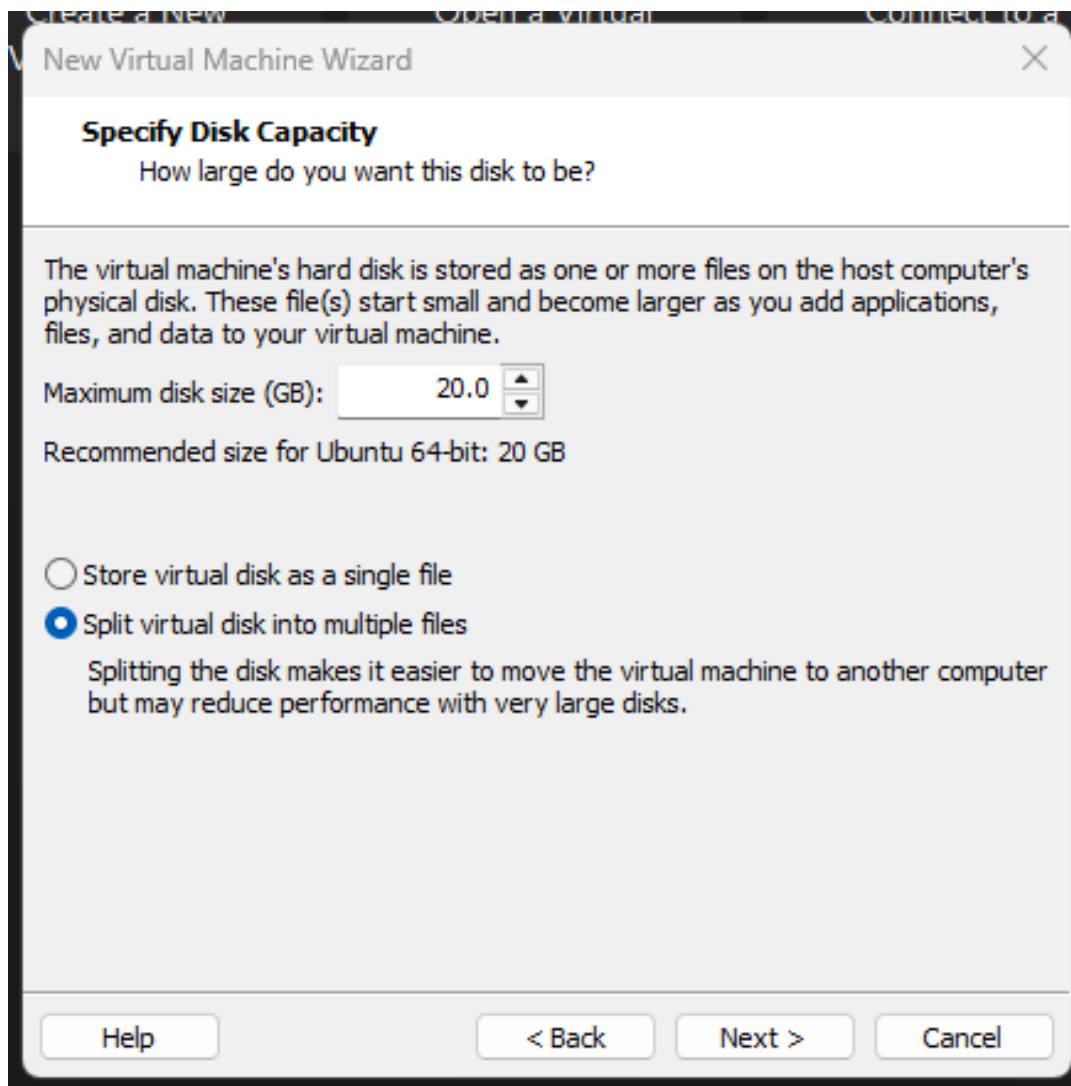
Phiên bản Hadoop mà em dùng là Hadoop 3.5.0.



Hình 3: Điền thông tin cho máy ảo.



Hình 4: Chọn vị trí lưu trữ máy ảo.



Hình 5: Chọn dung lượng ổ cứng 20 GB cho máy ảo.

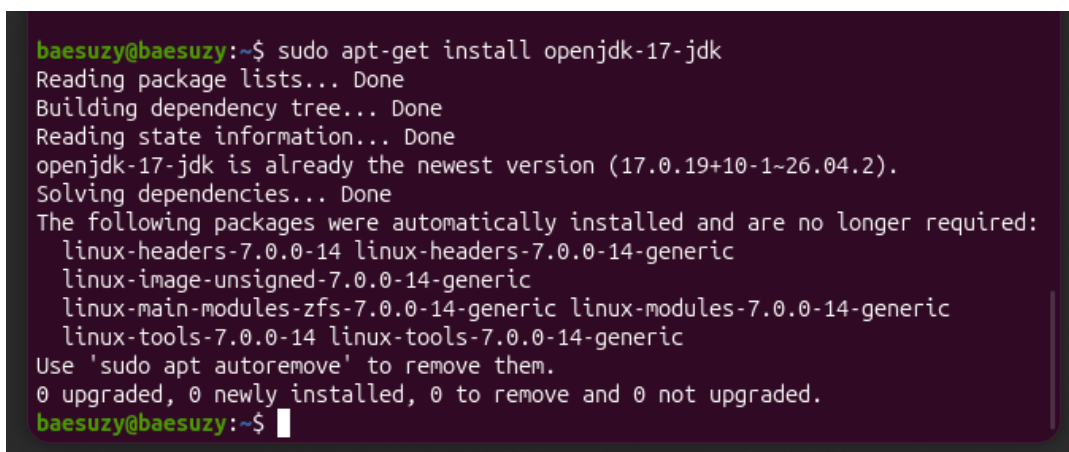
2.2.1 Cài đặt Java phù hợp với phiên bản Hadoop

Theo trang Hadoop Java Versions [2] đề cập thì Hadoop 3.5.0 yêu cầu JDK 17 nên em sẽ cài đặt JDK 17 trước. Hadoop chạy trên nền Java, nên các thành phần như NameNode, DataNode và các lệnh Hadoop đều cần JVM để hoạt động. Với Hadoop 3.5.0, phía server yêu cầu JDK 17, vì vậy nếu cài sai phiên bản Java thì có thể gặp lỗi khi chạy Hadoop hoặc khi khởi động các daemon. Do đó em cài JDK 17 trước để đảm bảo đúng với phiên bản Hadoop đang sử dụng.

Chạy lệnh sau để cài đặt JDK 17:

```
1 sudo apt-get install openjdk-17-jdk
```

Quá trình cài đặt như Hình 6.



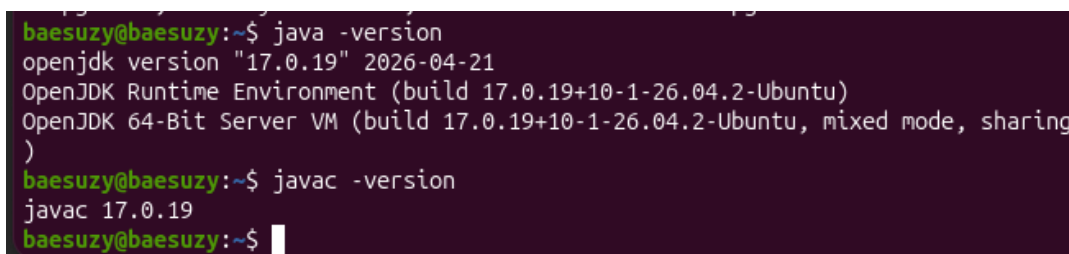
```
baesuzy@baesuzy:~$ sudo apt-get install openjdk-17-jdk
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openjdk-17-jdk is already the newest version (17.0.19+10-1-26.04.2).
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14 linux-headers-7.0.0-14-generic
  linux-image-unsigned-7.0.0-14-generic
  linux-main-modules-zfs-7.0.0-14-generic linux-modules-7.0.0-14-generic
  linux-tools-7.0.0-14 linux-tools-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
baesuzy@baesuzy:~$
```

Hình 6: Cài đặt OpenJDK 17.

Kiểm tra phiên bản Java đã cài đặt bằng lệnh:

```
1 java -version
2 javac -version
```

Kết quả như Hình 7.



```
baesuzy@baesuzy:~$ java -version
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment (build 17.0.19+10-1-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.19+10-1-26.04.2-Ubuntu, mixed mode, sharing)
baesuzy@baesuzy:~$ javac -version
javac 17.0.19
baesuzy@baesuzy:~$
```

Hình 7: Kiểm tra phiên bản Java và Java compiler.

2.2.2 Cài đặt SSH

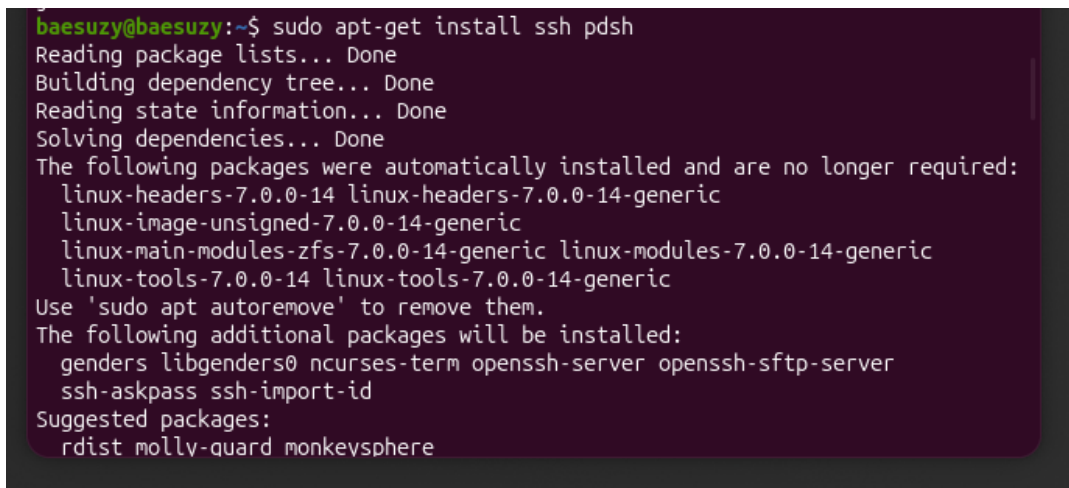
Hadoop sử dụng các script như `start-dfs.sh` và `stop-dfs.sh` để khởi động hoặc dừng các daemon thông qua SSH. Dù ở chế độ Pseudo mode chỉ chạy trên một máy, Hadoop vẫn xem localhost

như một node trong cluster và cần SSH để gọi các tiến trình như NameNode, DataNode. Còn `pdsh` là công cụ hỗ trợ chạy lệnh SSH song song và được Hadoop khuyến nghị cài thêm để quản lý việc SSH tốt hơn.

Trang hướng dẫn của Hadoop Apache cũng yêu cầu cài đặt `ssh` để Hadoop có thể vận hành được. Thêm vào đó khuyến nghị cài thêm `pdsh` để quản lý `ssh` tốt hơn.

```
1 sudo apt-get install ssh pdsh
```

Hình ảnh bước cài đặt như Hình 8.



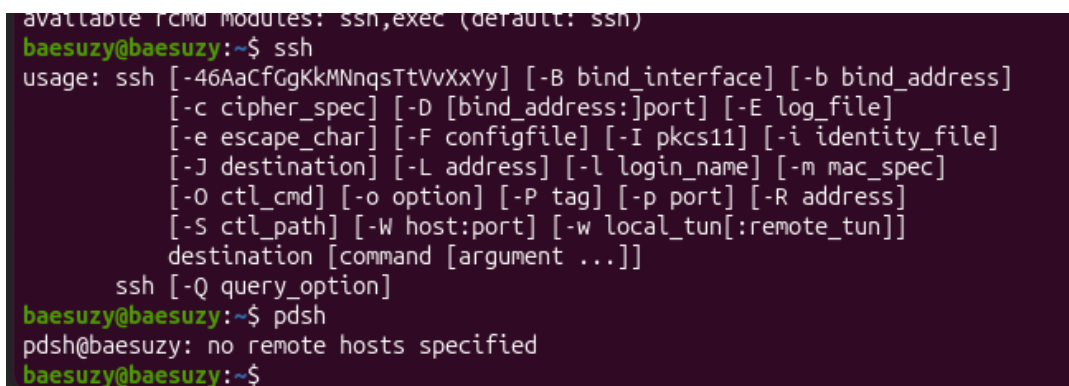
```
baesuzy@baesuzy:~$ sudo apt-get install ssh pdsh
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14 linux-headers-7.0.0-14-generic
  linux-image-unsigned-7.0.0-14-generic
  linux-main-modules-zfs-7.0.0-14-generic linux-modules-7.0.0-14-generic
  linux-tools-7.0.0-14 linux-tools-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  genders libgenders0 ncurses-term openssh-server openssh-sftp-server
  ssh-askpass ssh-import-id
Suggested packages:
  rdist molly-guard monkeysphere
```

Hình 8: Cài đặt `ssh` và `pdsh`.

Kiểm tra xem các package đã được cài đặt chưa bằng lệnh:

```
1 ssh
2 pdsh
```

Kết quả như Hình 9.

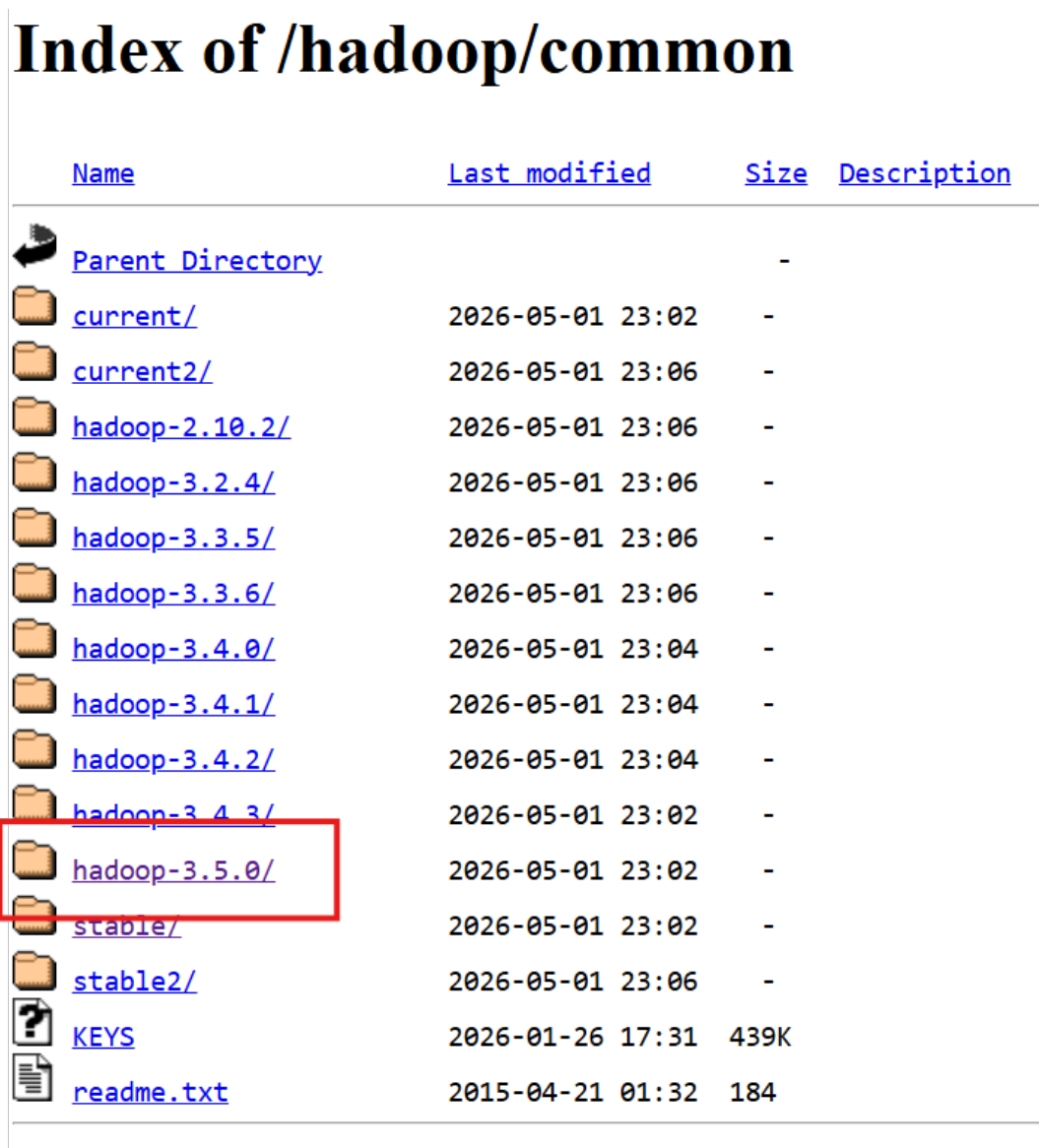


```
available rcmd modules: ssh,exec (default: ssh)
baesuzy@baesuzy:~$ ssh
usage: ssh [-46AaCfGgKkMnNqsTtVvXxYy] [-B bind_interface] [-b bind_address]
  [-c cipher_spec] [-D [bind_address:]port] [-E log_file]
  [-e escape_char] [-F configfile] [-I pkcs11] [-i identity_file]
  [-J destination] [-L address] [-l login_name] [-m mac_spec]
  [-O ctl_cmd] [-o option] [-P tag] [-p port] [-R address]
  [-S ctl_path] [-W host:port] [-w local_tun[:remote_tun]]
  destination [command [argument ...]]
  ssh [-Q query_option]
baesuzy@baesuzy:~$ pdsh
pdsh@baesuzy: no remote hosts specified
baesuzy@baesuzy:~$
```

Hình 9: Kiểm tra `ssh` và `pdsh` đã được cài đặt.

2.2.3 Tải và cài đặt Hadoop

Tải về Hadoop 3.5.0. Trước tiên, để tải về Hadoop 3.5.0 cần truy cập vào kho lưu trữ Hadoop Common [3] rồi chọn `hadoop-3.5.0` (Hình 10).



Name	Last modified	Size	Description
 Parent Directory		-	
 current/	2026-05-01 23:02	-	
 current2/	2026-05-01 23:06	-	
 hadoop-2.10.2/	2026-05-01 23:06	-	
 hadoop-3.2.4/	2026-05-01 23:06	-	
 hadoop-3.3.5/	2026-05-01 23:06	-	
 hadoop-3.3.6/	2026-05-01 23:06	-	
 hadoop-3.4.0/	2026-05-01 23:04	-	
 hadoop-3.4.1/	2026-05-01 23:04	-	
 hadoop-3.4.2/	2026-05-01 23:04	-	
 hadoop-3.4.3/	2026-05-01 23:02	-	
 hadoop-3.5.0/	2026-05-01 23:02	-	
 stable/	2026-05-01 23:02	-	
 stable2/	2026-05-01 23:06	-	
 KEYS	2026-01-26 17:31	439K	
 readme.txt	2015-04-21 01:32	184	

Hình 10: Kho lưu trữ Hadoop Common.

Trong thư mục `hadoop-3.5.0`, chúng ta sẽ có nhiều lựa chọn để tải về như Hình 11 nhưng để đơn giản thì em sẽ chọn phiên bản binary `hadoop-3.5.0.tar.gz` để tải về.

Để tải về trên máy ảo Ubuntu, dùng lệnh:

```
1 wget https://dlcdn.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
```

Quá trình tải về như Hình 12.

Sau khi tải về xong, giải nén file `hadoop-3.5.0.tar.gz` bằng lệnh:

Index of /hadoop/common/hadoop-3.5.0

Name	Last modified	Size	Description
 Parent Directory		-	
 CHANGELOG.md	2026-04-01 20:43	144K	
 CHANGELOG.md.asc	2026-04-01 20:43	833	
 CHANGELOG.md.sha512	2026-04-01 20:43	153	
 RELEASENOTES.md	2026-04-01 20:43	14K	
 RELEASENOTES.md.asc	2026-04-01 20:43	833	
 RELEASENOTES.md.sha512	2026-04-01 20:43	156	
 hadoop-3.5.0-aarch64.tar.gz	2026-04-01 20:43	554M	
 hadoop-3.5.0-aarch64.tar.gz.asc	2026-04-01 20:43	833	
 hadoop-3.5.0-aarch64.tar.gz.sha512	2026-04-01 20:43	160	
 hadoop-3.5.0-rat.txt	2026-04-01 20:43	2.3M	
 hadoop-3.5.0-rat.txt.asc	2026-04-01 20:43	833	
 hadoop-3.5.0-rat.txt.sha512	2026-04-01 20:43	161	
 hadoop-3.5.0-site.tar.gz	2026-04-01 20:43	52M	
 hadoop-3.5.0-site.tar.gz.asc	2026-04-01 20:43	833	
 hadoop-3.5.0-site.tar.gz.sha512	2026-04-01 20:43	165	
 hadoop-3.5.0-src.tar.gz	2026-04-01 20:43	39M	
 hadoop-3.5.0-src.tar.gz.asc	2026-04-01 20:43	833	
 hadoop-3.5.0-src.tar.gz.sha512	2026-04-01 20:43	164	
 hadoop-3.5.0.tar.gz	2026-04-01 20:43	554M	
 hadoop-3.5.0.tar.gz.asc	2026-04-01 20:43	833	
 hadoop-3.5.0.tar.gz.sha512	2026-04-01 20:43	160	

Hình 11: Các bản phát hành trong thư mục hadoop-3.5.0.

```
baesuzzy@baesuzzy:~$ wget https://dlcdn.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
--2026-07-05 16:05:23-- https://dlcdn.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 581280703 (554M) [application/x-gzip]
Saving to: 'hadoop-3.5.0.tar.gz'

hadoop-3.5.0.tar.gz 100%[=====] 554.35M 83.5MB/s in 7.1s

2026-07-05 16:05:30 (78.6 MB/s) - 'hadoop-3.5.0.tar.gz' saved [581280703/581280703]

baesuzzy@baesuzzy:~$
```

Hình 12: Tải Hadoop 3.5.0 bằng wget.

```
1 tar -xvzf hadoop-3.5.0.tar.gz
```

Quá trình giải nén như Hình 13.

Như vậy là em đã tải về thành công Hadoop 3.5.0 và tiếp theo là cần setup cấu hình để Hadoop có thể hoạt động.

Cấu hình Hadoop. Ở step trước, em đã giải nén Hadoop 3.5.0 ở home nên sẽ có 1 thư mục là `hadoop-3.5.0`. Bây giờ, em sẽ `cd` vào thư mục `hadoop-3.5.0` (để tiện thao tác sau này) và `ls` để xem các file và thư mục bên trong.

```
1 cd hadoop-3.5.0
2 ls
```

Kết quả như Hình 14.

Bước này, em cần chỉnh biến môi trường `JAVA_HOME` trong file `etc/hadoop/hadoop-env.sh` để Hadoop biết được đường dẫn của thư mục chứa `java` hay `javac`. Biến `JAVA_HOME` dùng để chỉ cho Hadoop biết thư mục cài đặt Java nằm ở đâu. Khi chạy các lệnh Hadoop, hệ thống cần tìm đúng Java để khởi động các tiến trình của Hadoop. Nếu không cấu hình `JAVA_HOME`, Hadoop có thể không tìm thấy Java hoặc dùng sai phiên bản Java, dẫn đến lỗi khi chạy. Để biết được đường dẫn đó em dùng câu lệnh:

```
1 readlink -f $(which java)
```

Giải thích: `which java` sẽ trả về đường dẫn của file thực thi `java`, nhưng đây chỉ là một symlink. Để biết được đường dẫn thực sự của file `java`, em dùng `readlink -f` để theo dõi symbolic link đó. Kết quả như Hình 15.

Tới đây, em sẽ copy `/usr/lib/jvm/java-17-openjdk-amd64` và paste vào file `hadoop-env.sh` để chỉnh biến môi trường `JAVA_HOME`.

Mở file `hadoop-env.sh` bằng lệnh:

```
baesuzy@baesuzy: ~
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/external.png
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/breadcrumbs.jpg
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/logo_maven.jpg
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/bg.jpg
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/icon_success_sml.gif
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/expanded.gif
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/logos/
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/logos/build-by-maven-b
lack.png
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/logos/maven-feather.pn
g
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/logos/build-by-maven-w
hite.png
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/icon_info_sml.gif
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/images/apache-maven-project-2
.png
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/project-reports.html
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/dependency-analysis.html
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/css/
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/css/print.css
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/css/site.css
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/css/maven-theme.css
hadoop-3.5.0/share/doc/hadoop/hadoop-maven-plugins/css/maven-base.css
baesuzy@baesuzy:~$
```

Hình 13: Giải nén Hadoop 3.5.0.

```
baesuzy@baesuzy:~$ cd hadoop-3.5.0/
baesuzy@baesuzy:~/hadoop-3.5.0$ ls
Dockerfile      NOTICE-binary  bin           include       licenses-binary
LICENSE-binary  NOTICE.txt     compose       lib           sbin
LICENSE.txt     README.txt     etc          libexec       share
baesuzy@baesuzy:~/hadoop-3.5.0$
```

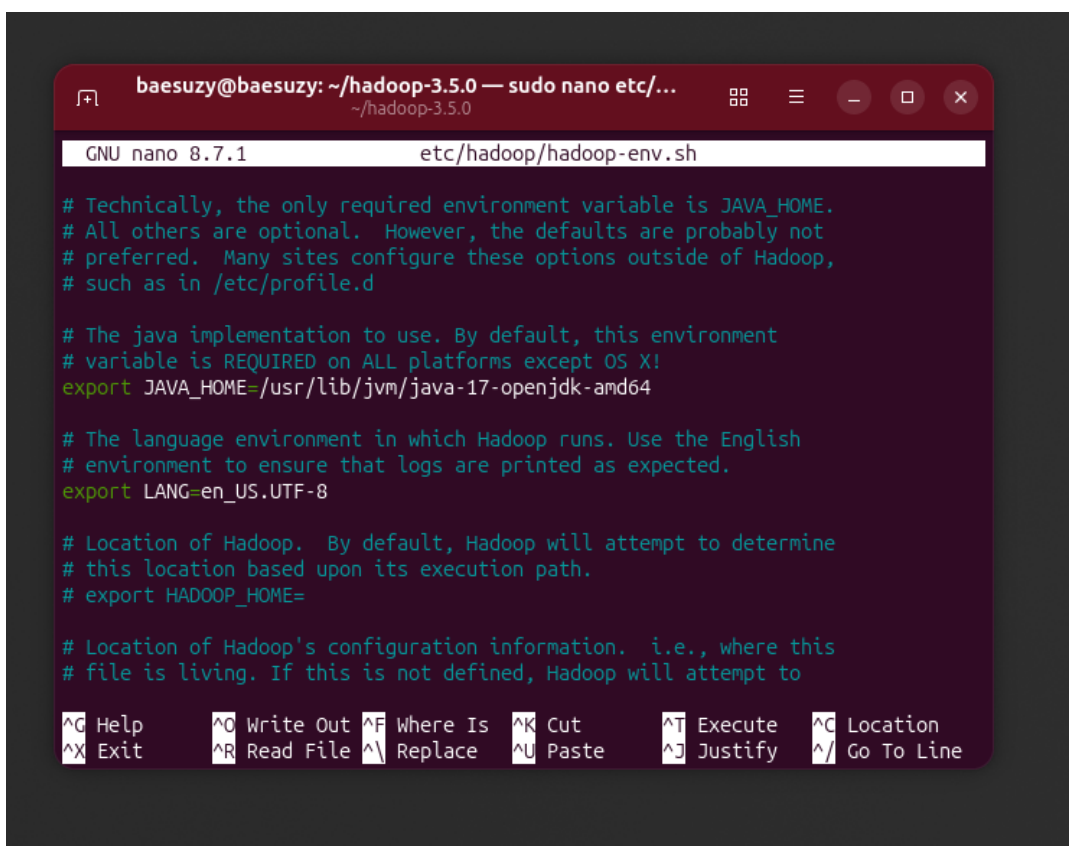
Hình 14: Nội dung thư mục hadoop-3.5.0.

```
baesuzy@baesuzy:~/hadoop-3.5.0$ readlink -f $(which java)
/usr/lib/jvm/java-17-openjdk-amd64/bin/java
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 15: Tìm đường dẫn thực của java bằng readlink -f.


```
1 sudo nano etc/hadoop/hadoop-env.sh
```

Chỉnh sửa như Hình 16.



Hình 16: Khai báo JAVA_HOME trong `hadoop-env.sh`.

Sau đó, nhấn `Ctrl + O` và `Enter` để lưu file và `Ctrl + X` để thoát khỏi nano.

Cuối cùng thử câu lệnh `bin/hadoop` để kiểm tra xem Hadoop đã cài đặt thành công hay chưa (Hình 17).

Như vậy là em đã cài đặt xong Hadoop 3.5.0. Tiếp theo em sẽ tiến hành cấu hình Hadoop để chạy ở chế độ Pseudo mode.

2.2.4 Cấu hình Pseudo-distributed mode trên Hadoop

Chỉnh sửa file `core-site.xml`. File `core-site.xml` là file cấu hình chung của Hadoop. File này chứa các thông số nền tảng mà nhiều thành phần Hadoop cùng sử dụng, ví dụ như địa chỉ filesystem mặc định. Trong bước này, em chỉnh file này để Hadoop biết khi thao tác với filesystem thì sẽ dùng HDFS đang chạy trên máy local.

Chỉnh sửa file `core-site.xml` để cấu hình Hadoop chạy ở chế độ Pseudo mode. Mở file bằng lệnh:

```
1 sudo nano etc/hadoop/core-site.xml
```

Sau đó thêm vào đoạn cấu hình sau:

```
baesuzzy@baesuzzy:~/hadoop-3.5.0$ sudo nano etc/hadoop/hadoop-env.sh
baesuzzy@baesuzzy:~/hadoop-3.5.0$ bin/hadoop
Usage: hadoop [OPTIONS] SUBCOMMAND [SUBCOMMAND OPTIONS]
or      hadoop [OPTIONS] CLASSNAME [CLASSNAME OPTIONS]
       where CLASSNAME is a user-provided Java class

OPTIONS is none or any of:

buildpaths          attempt to add class files from build tree
--config dir        Hadoop config directory
--debug             turn on shell script debug mode
--help              usage information
hostnames list[,of,host,names] hosts to use in worker mode
hosts filename       list of hosts to use in worker mode
loglevel level       set the log4j level for this command
workers             turn on worker mode

SUBCOMMAND is one of:

Admin Commands:
```

Hình 17: Kiểm tra Hadoop bằng lệnh bin/hadoop.

```
1 <configuration>
2   <property>
3     <name>fs.defaultFS</name>
4     <value>hdfs://localhost:9000</value>
5   </property>
6 </configuration>
```

Đoạn cấu hình này đặt `fs.defaultFS` là `hdfs://localhost:9000`. Điều này có nghĩa là filesystem mặc định của Hadoop sẽ là HDFS, với NameNode chạy trên máy `localhost` và lắng nghe ở port 9000. Nhờ vậy, khi em chạy các lệnh như `hdfs dfs -mkdir` hay `hdfs dfs -put`, Hadoop sẽ hiểu là thao tác trên HDFS này thay vì filesystem local.

Ghi lưu và thoát khỏi nano (Hình 18).

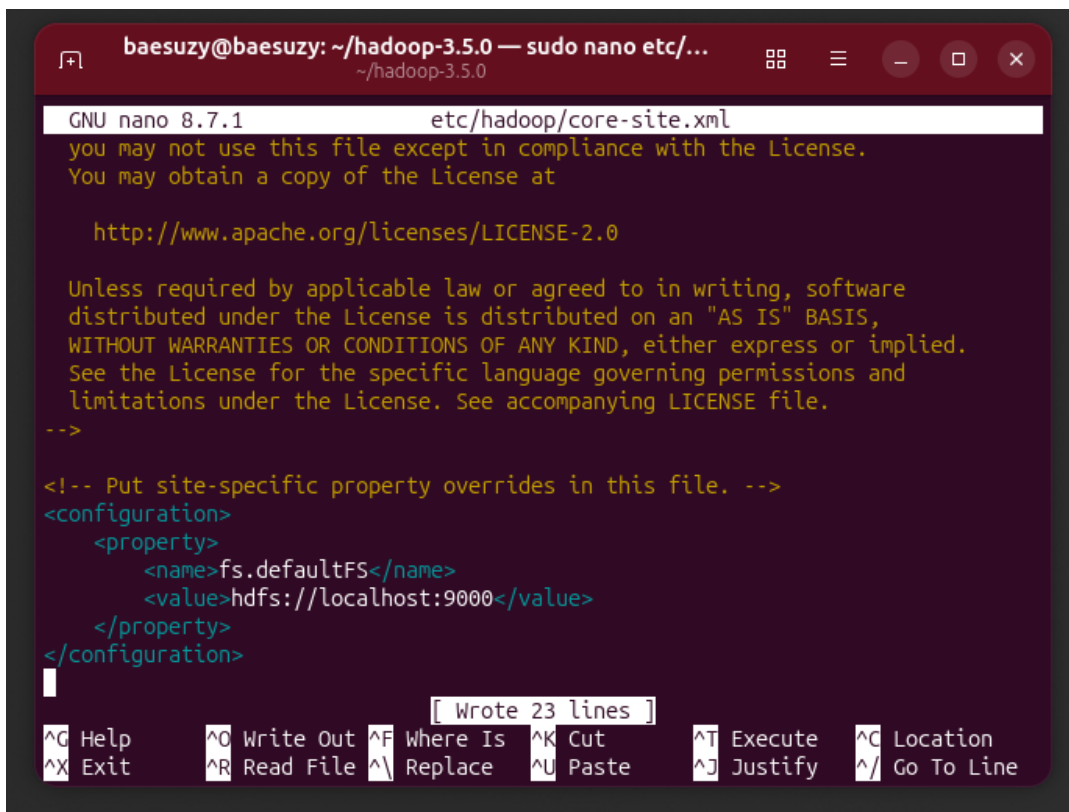
Chỉnh sửa file `hdfs-site.xml`. File `hdfs-site.xml` là file cấu hình riêng cho HDFS. File này dùng để thiết lập các thông số liên quan đến NameNode, DataNode, replication, thư mục lưu dữ liệu và các cấu hình khác của hệ thống file phân tán Hadoop. Ở đây em chỉnh file này để cấu hình cách HDFS lưu dữ liệu trong chế độ Pseudo mode.

Tiếp theo chỉnh sửa file `hdfs-site.xml` để cấu hình các thông số liên quan đến HDFS. Mở file bằng lệnh:

```
1 sudo nano etc/hadoop/hdfs-site.xml
```

Sau đó thêm vào đoạn cấu hình sau:

```
1 <configuration>
```

Hình 18: Cấu hình core-site.xml cho Pseudo mode.

```

2     <property>
3         <name>dfs.replication</name>
4         <value>1</value>
5     </property>
6 </configuration>

```

Thuộc tính `dfs.replication` quy định số bản sao của mỗi block dữ liệu trong HDFS. Khi đặt giá trị là 1, mỗi block chỉ có một bản sao. Điều này phù hợp với Pseudo mode vì em chỉ chạy Hadoop trên một máy, nếu để replication lớn hơn 1 thì HDFS không có đủ DataNode để tạo thêm bản sao. Ghi lưu và thoát khỏi nano (Hình 19).

Cài đặt passphraseless ssh. Giải thích: Trong Pseudo mode, Hadoop vẫn khởi động các daemon thông qua SSH, dù tất cả đều chạy trên cùng một máy. Vì vậy Hadoop cần SSH vào `localhost` để start hoặc stop các tiến trình như NameNode và DataNode. Nếu mỗi lần SSH đều yêu cầu nhập mật khẩu thì các script khởi động Hadoop sẽ bị gián đoạn. Do đó cần thiết lập passphraseless SSH để máy có thể SSH vào chính nó mà không cần nhập mật khẩu.

Để làm được điều này, em sẽ tạo một cặp khóa SSH và thêm khóa công khai vào file `authorized_keys`.

```

1 ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
2 cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
3 chmod 0600 ~/.ssh/authorized_keys

```

The screenshot shows a terminal window with the title bar "baesuzu@baesuzu: ~/hadoop-3.5.0 — sudo nano etc/...". The window displays the content of the file `etc/hadoop/hdfs-site.xml` in the nano editor. The text in the editor includes the GNU nano version (8.7.1), a license notice pointing to <http://www.apache.org/licenses/LICENSE-2.0>, and an XML configuration block for `dfs.replication` set to 1. The configuration block is as follows:

```
<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```

At the bottom of the terminal window, there is a status bar with various keyboard shortcuts for nano editor operations:

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line

Hình 19: Cấu hình `hdfs-site.xml` cho Pseudo mode.

Kết quả như Hình 20.

```
baesuzy@baesuzy:~/hadoop-3.5.0$ ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
Generating public/private rsa key pair.
Your identification has been saved in /home/baesuzy/.ssh/id_rsa
Your public key has been saved in /home/baesuzy/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:nWWgsWIhXpAYUq6+kn7L0Cask8seC9iEtVfEVT00594 baesuzy@baesuzy
The key's randomart image is:
+---[RSA 3072]-----+
|o++oo.....o|
|oo.o o. o .|
| + . o * o|
| + . o = = +|
|o o . . S o|
|+o.. o .|
|oO.o . E|
|O O.|
|*O.o.|
+---[SHA256]-----+
baesuzy@baesuzy:~/hadoop-3.5.0$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
baesuzy@baesuzy:~/hadoop-3.5.0$ chmod 0600 ~/.ssh/authorized_keys
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 20: Tạo cặp khóa SSH và thêm public key vào `authorized_keys`.

Lệnh `ssh-keygen` tạo ra một cặp khóa gồm private key và public key. Private key được giữ ở máy hiện tại, còn public key được thêm vào file `~/.ssh/authorized_keys`. Khi SSH vào `localhost`, SSH server sẽ kiểm tra xem public key trong `authorized_keys` có khớp với private key của user hay không. Nếu khớp thì cho phép đăng nhập mà không cần nhập mật khẩu. Lệnh `chmod 0600` dùng để giới hạn quyền truy cập file `authorized_keys`, vì SSH yêu cầu file này không được cấp quyền quá rộng.

Giờ em sẽ thử SSH vào `localhost` để kiểm tra xem có cần nhập mật khẩu không:

```
1 ssh localhost
```

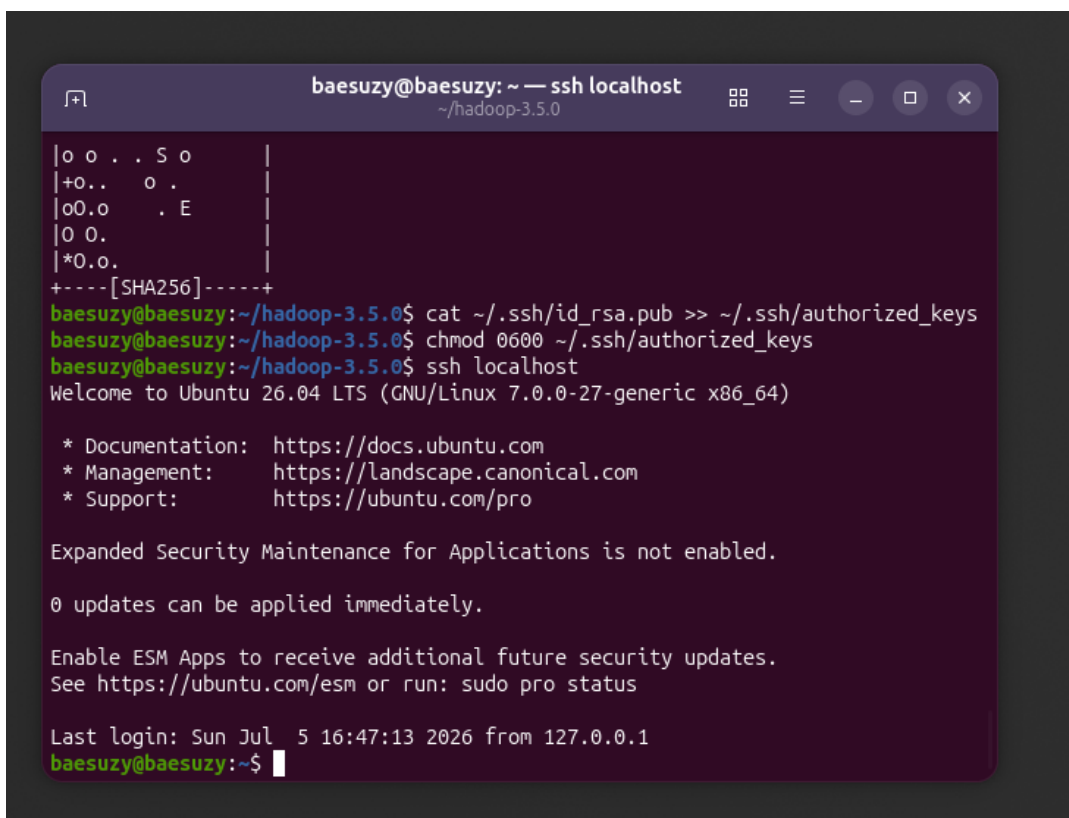
Kết quả như Hình 21.

Như vậy là em đã thiết lập thành công passphraseless SSH.

2.2.5 Khởi động HDFS

Giải thích: Sau khi cấu hình xong, HDFS vẫn chưa hoạt động ngay mà cần khởi động các daemon của nó. Khi chạy `start-dfs.sh`, Hadoop sẽ khởi động NameNode và DataNode. NameNode quản lý metadata của hệ thống file, còn DataNode lưu dữ liệu thực tế. Nếu chưa khởi chạy HDFS thì các lệnh thao tác với HDFS sẽ không kết nối được tới NameNode.

Lệnh `bin/hdfs namenode -format` dùng để khởi tạo filesystem metadata cho NameNode ở lần chạy đầu tiên. Có thể hiểu bước này giống như tạo cấu trúc ban đầu cho HDFS để NameNode có nơi lưu thông tin về thư mục, file và block dữ liệu. Lệnh này chỉ nên chạy lần đầu khi thiết lập HDFS mới, vì nếu format lại sau khi đã có dữ liệu thì metadata cũ có thể bị mất.



Hình 21: SSH vào localhost không cần nhập mật khẩu.

Trước tiên phải chạy lệnh format hdfs ở lần đầu:

```
1 bin/hdfs namenode -format
```

Kết quả như Hình 22.

Sau đó, khởi động HDFS bằng lệnh:

```
1 sbin/start-dfs.sh
```

Kết quả như Hình 23.

Khi đã chạy thành công thì chúng ta có thể truy cập vào giao diện web của HDFS bằng cách mở trình duyệt và truy cập vào địa chỉ <http://localhost:9870/>. Nếu thấy giao diện HDFS hiện ra thì chứng tỏ HDFS đã được khởi động thành công (Hình 24).

Giao diện Overview cũng cho thấy localhost:9000 (active), như vậy là NameNode đã hoạt động đúng port mà trước đó em đã cấu hình.

2.3 Thao tác với Hadoop

Sau khi cài đặt xong Hadoop và đã khởi động HDFS, giờ em sẽ thực hiện các yêu cầu trong Lab 1.

```

baesuzy@baesuzy: ~/hadoop-3.5.0
~ /hadoop-3.5.0

Last login: Sun Jul  5 16:47:13 2026 from 127.0.0.1
baesuzy@baesuzy:~$ exit
logout
Connection to localhost closed.
baesuzy@baesuzy:~/hadoop-3.5.0$ bin/hdfs namenode -format
WARNING: /home/baesuzy/hadoop-3.5.0/logs does not exist. Creating.
2026-07-05 19:15:06,880 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG:  host = baesuzy/127.0.1.1
STARTUP_MSG:  args = [-format]
STARTUP_MSG:  version = 3.5.0
STARTUP_MSG:  classpath = /home/baesuzy/hadoop-3.5.0/etc/hadoop:/home/baesuzy/h
adoop-3.5.0/share/hadoop/common/lib/jsr305-3.0.2.jar:/home/baesuzy/hadoop-3.5.0/shar
e/hadoop/common/lib/curator-client-5.2.0.jar:/home/baesuzy/hadoop-3.5.0/shar
e/hadoop/common/lib/hk2-utils-2.6.1.jar:/home/baesuzy/hadoop-3.5.0/share/hadoop/
common/lib/jetty-util-9.4.58.v20250814.jar:/home/baesuzy/hadoop-3.5.0/share/hadoo
re/hadoop/lib/hadoop-auth-3.5.0.jar:/home/baesuzy/hadoop-3.5.0/share/hadoop/
common/lib/osgi-resource-locator-1.0.3.jar:/home/baesuzy/hadoop-3.5.0/share/
hadoop/common/lib/commons-cli-1.9.0.jar:/home/baesuzy/hadoop-3.5.0/share/hadoop/
common/lib/commons-daemon-1.0.13.jar:/home/baesuzy/hadoop-3.5.0/share/hadoop/com
mon/lib/httpcore-4.4.13.jar:/home/baesuzy/hadoop-3.5.0/share/hadoop/common/lib/k
erby-pkix-2.0.3.jar:/home/baesuzy/hadoop-3.5.0/share/hadoop/common/lib/commons-c
odec-1.15.jar:/home/baesuzy/hadoop-3.5.0/share/hadoop/common/lib/netty-handler-4
1.120.5.jar:/home/baesuzy/hadoop-3.5.0/share/hadoop/common/lib/azkaban-common-

```

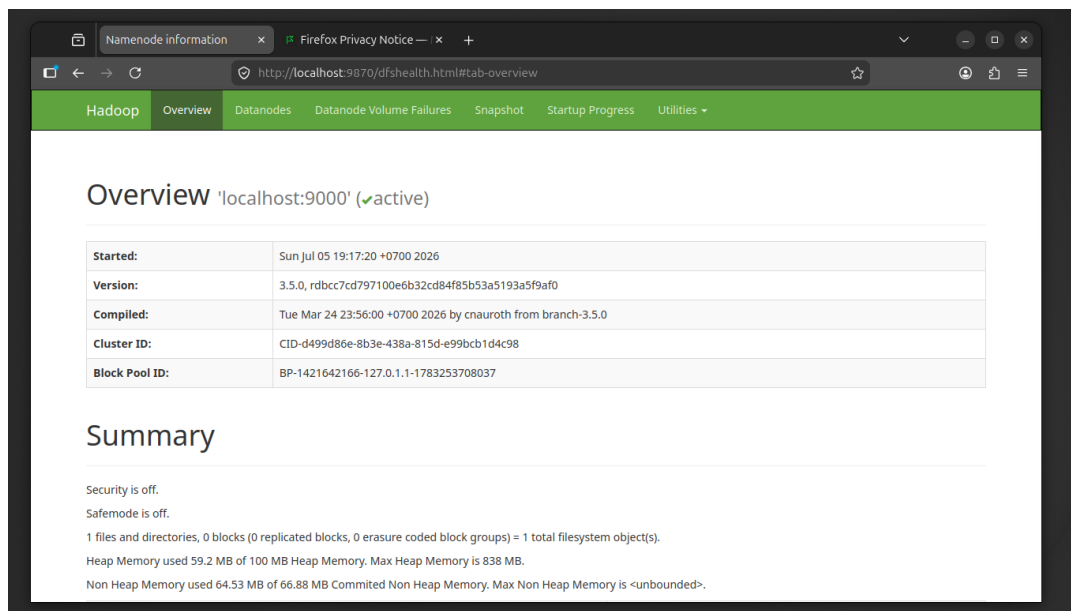
Hình 22: Format HDFS lần đầu bằng `bin/hdfs namenode -format`.

```

baesuzy@baesuzy:~/hadoop-3.5.0$ sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [baesuzy]
baesuzy: Warning: Permanently added 'baesuzy' (ED25519) to the list of known hos
ts.
baesuzy@baesuzy:~/hadoop-3.5.0$

```

Hình 23: Khởi động HDFS bằng `sbin/start-dfs.sh`.



Hình 24: Giao diện web của HDFS.

2.3.1 Task 1: Tạo một thư mục /hcmus trên HDFS

Để tạo thư mục /hcmus trên HDFS, em sử dụng lệnh:

```
1 bin/hdfs dfs -mkdir /hcmus
```

Lệnh này sẽ tạo một folder tên hcmus tại root directory của HDFS. Để kiểm tra xem thư mục đã được tạo thành công hay chưa, em có thể liệt kê các thư mục trong HDFS bằng lệnh:

```
1 bin/hdfs dfs -ls /
```

Kết quả như Hình 25.

Như vậy là đã tạo thành công thư mục /hcmus trên HDFS.

2.3.2 Task 2: Tạo một user với tên khtn_23127150

Để tạo một user mới trên hệ thống Ubuntu, em sử dụng lệnh adduser. Lệnh này sẽ tạo một user mới và thiết lập thư mục home cho user đó. Cụ thể, để tạo user khtn_23127150, em chạy lệnh:

```
1 sudo adduser khtn_23127150
```

Khi Enter lệnh này thì Ubuntu sẽ yêu cầu nhập mật khẩu và các thông tin khác cho user mới. Em nhập mật khẩu và các thông tin theo yêu cầu. Sau khi hoàn tất, user khtn_23127150 đã được tạo thành công (Hình 26).

Em sẽ thử đăng nhập vào user mới tạo bằng lệnh:

```
1 su - khtn_23127150
```

Kết quả như Hình 27.

Như vậy là em đã tạo thành công user khtn_23127150 trên Ubuntu.

```
baesuzy@baesuzy: ~/hadoop-3.5.0
~/hadoop-3.5.0
baesuzy@baesuzy:~/hadoop-3.5.0$ bin/hdfs dfs -mkdir /hcmus
baesuzy@baesuzy:~/hadoop-3.5.0$ bin/hdfs dfs -ls /
Found 1 items
drwxr-xr-x  - baesuzy supergroup          0 2026-07-05 20:06 /hcmus
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 25: Tạo và liệt kê thư mục /hcmus trên HDFS.

```
baesuzy@baesuzy:~/hadoop-3.5.0$ sudo adduser khtn_23127150
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
Changing the user information for khtn_23127150
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n]
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 26: Tạo user khtn_23127150 bằng adduser.

```
Is the information correct? [Y/n]
baesuzy@baesuzy:~/hadoop-3.5.0$ su - khtn_23127150
Password:
khtn_23127150@baesuzy:~$
```

Hình 27: Đăng nhập vào user khtn_23127150.

2.3.3 Task 3: Tạo subfolder /hcmus/23127150 trên HDFS và upload một file vào đó

Tạo một file text với nội dung baesuzzy lưu vào /tmp bằng lệnh:

```
1 echo "baesuzzy" > /tmp/baesuzzy.txt
```

Sau đó, tạo folder con /hcmus/23127150 trên HDFS bằng lệnh:

```
1 bin/hdfs dfs -mkdir /hcmus/23127150
```

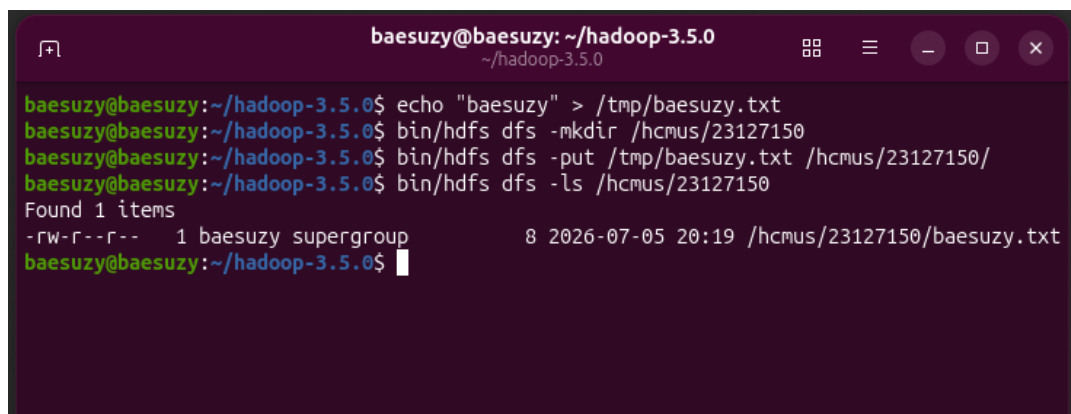
Sau đó, upload file baesuzzy.txt vào folder con này bằng lệnh:

```
1 bin/hdfs dfs -put /tmp/baesuzzy.txt /hcmus/23127150/
```

Kiểm tra xem file đã được upload thành công hay chưa bằng lệnh:

```
1 bin/hdfs dfs -ls /hcmus/23127150
```

Kết quả như Hình 28.



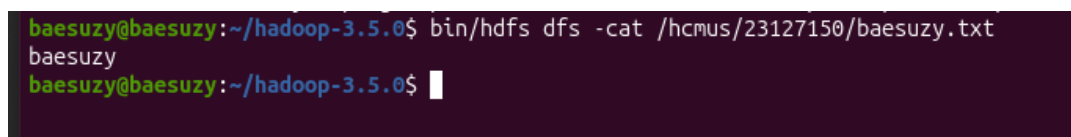
```
baesuzzy@baesuzzy: ~/hadoop-3.5.0
~/hadoop-3.5.0
baesuzzy@baesuzzy:~/hadoop-3.5.0$ echo "baesuzzy" > /tmp/baesuzzy.txt
baesuzzy@baesuzzy:~/hadoop-3.5.0$ bin/hdfs dfs -mkdir /hcmus/23127150
baesuzzy@baesuzzy:~/hadoop-3.5.0$ bin/hdfs dfs -put /tmp/baesuzzy.txt /hcmus/23127150/
baesuzzy@baesuzzy:~/hadoop-3.5.0$ bin/hdfs dfs -ls /hcmus/23127150
Found 1 items
-rw-r--r--  1 baesuzzy supergroup      8 2026-07-05 20:19 /hcmus/23127150/baesuzzy.txt
baesuzzy@baesuzzy:~/hadoop-3.5.0$
```

Hình 28: Upload và liệt kê file trong /hcmus/23127150.

Như vậy đã đẩy được file baesuzzy.txt vào HDFS thành công. Kiểm tra xem nội dung file baesuzzy.txt có đúng không bằng lệnh:

```
1 bin/hdfs dfs -cat /hcmus/23127150/baesuzzy.txt
```

Kết quả như Hình 29.



```
baesuzzy@baesuzzy:~/hadoop-3.5.0$ bin/hdfs dfs -cat /hcmus/23127150/baesuzzy.txt
baesuzzy
baesuzzy@baesuzzy:~/hadoop-3.5.0$
```

Hình 29: Nội dung file baesuzzy.txt trên HDFS.

Nội dung file hiển thị đúng là baesuzzy, chứng tỏ file đã được upload thành công và nội dung không bị thay đổi.

2.3.4 Task 4: chmod 744 và đổi owner của /hcmus/23127150 thành khtn_23127150

Để thay đổi quyền truy cập của thư mục /hcmus/23127150 trên HDFS, em sử dụng lệnh chmod. Lệnh này sẽ thiết lập quyền truy cập cho thư mục theo chế độ 744, nghĩa là:

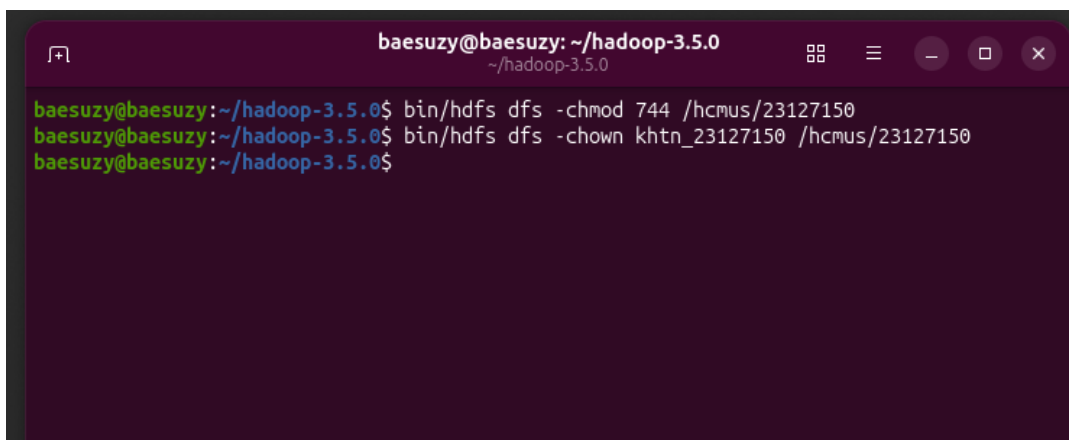
- Chủ sở hữu (owner) có quyền đọc, ghi và thực thi (7).
- Nhóm (group) có quyền đọc (4).
- Người khác (others) có quyền đọc (4).

```
1 bin/hdfs dfs -chmod 744 /hcmus/23127150
```

Để thay đổi chủ sở hữu (owner) của thư mục /hcmus/23127150 thành user khtn_23127150, em sử dụng lệnh chown. Lệnh này sẽ thay đổi chủ sở hữu của thư mục trên HDFS.

```
1 bin/hdfs dfs -chown khtn_23127150 /hcmus/23127150
```

Kết quả như Hình 30.



Hình 30: Đổi quyền và owner của /hcmus/23127150.

Kiểm tra lại quyền truy cập và chủ sở hữu của thư mục /hcmus/23127150 bằng lệnh:

```
1 bin/hdfs dfs -ls /hcmus
```

Kết quả như Hình 31.

Chỗ quyền truy cập hiển thị là drwxr-r-, nghĩa là quyền đã được thiết lập thành công. Chủ sở hữu của thư mục là khtn_23127150, chứng tỏ việc thay đổi owner cũng đã thành công.

Còn đây là trên giao diện <http://localhost:9870/explorer.html#/hcmus> (Hình 32):

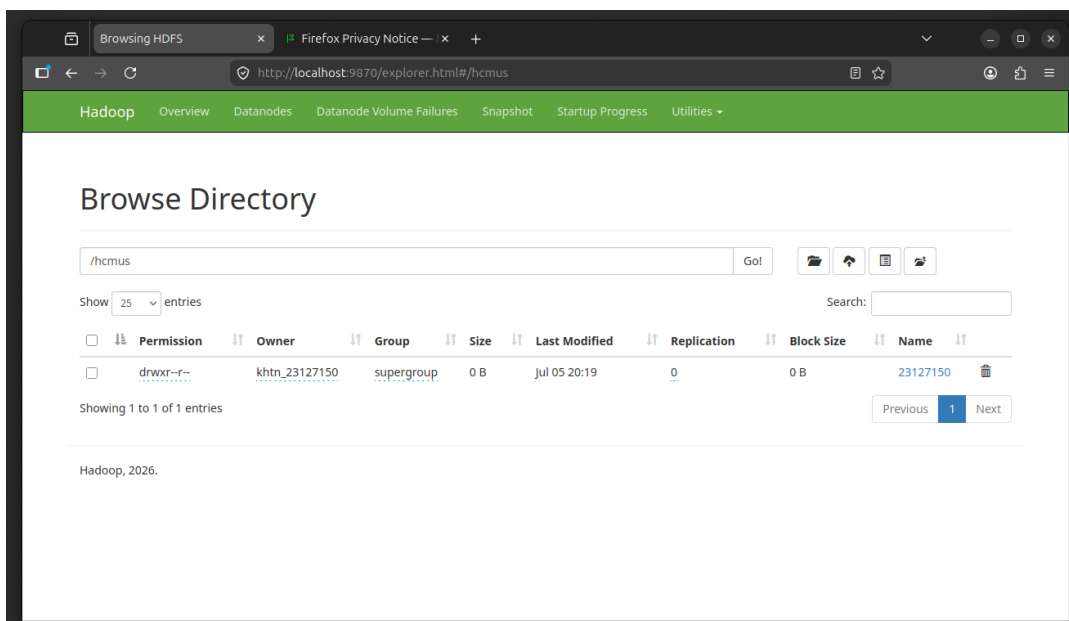
2.4 Chạy file JAR verify

Trước tiên, em sẽ back về thư mục home trên Ubuntu để làm nơi đặt file JAR verify bằng cd ~. Em vào drive để tải file hadoop-test.jar về và đặt ở ~/hadoop-test.jar.

Chạy lệnh java theo yêu cầu của bài lab:

```
baesuzzy@baesuzzy: ~/hadoop-3.5.0
~/hadoop-3.5.0
baesuzzy@baesuzzy:~/hadoop-3.5.0$ bin/hdfs dfs -chmod 744 /hcmus/23127150
baesuzzy@baesuzzy:~/hadoop-3.5.0$ bin/hdfs dfs -chown khtn_23127150 /hcmus/23127150
baesuzzy@baesuzzy:~/hadoop-3.5.0$ bin/hdfs dfs -ls /hcmus
Found 1 items
drwxr--r-- - khtn_23127150 supergroup          0 2026-07-05 20:19 /hcmus/23127150
baesuzzy@baesuzzy:~/hadoop-3.5.0$
```

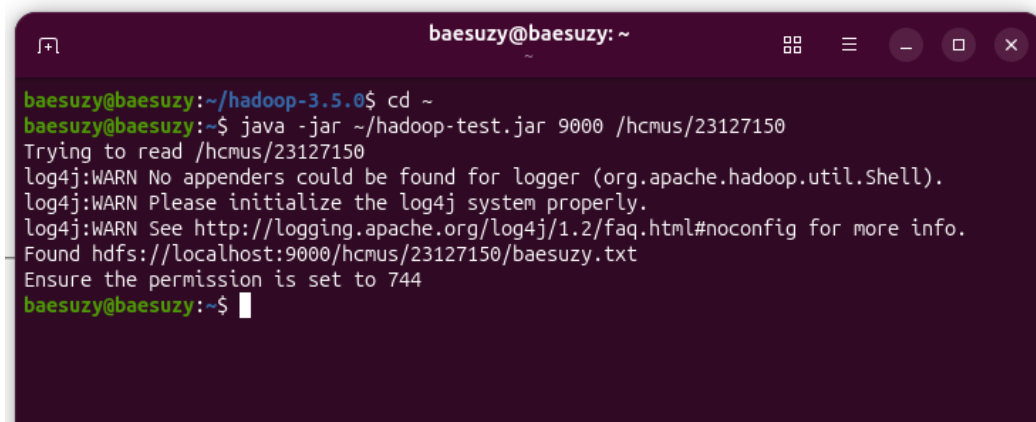
Hình 31: Quyền và owner của /hcmus/23127150 sau khi chỉnh.



Hình 32: Thư mục /hcmus trên giao diện web HDFS.

```
1 java -jar ~/hadoop-test.jar 9000 /hcmus/23127150
```

Kết quả chạy lần đầu như Hình 33.



```
baesuzy@baesuzy: ~
baesuzy@baesuzy:~/hadoop-3.5.0$ cd ~
baesuzy@baesuzy:~$ java -jar ~/hadoop-test.jar 9000 /hcmus/23127150
Trying to read /hcmus/23127150
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23127150/baesuzy.txt
Ensure the permission is set to 744
baesuzy@baesuzy:~$
```

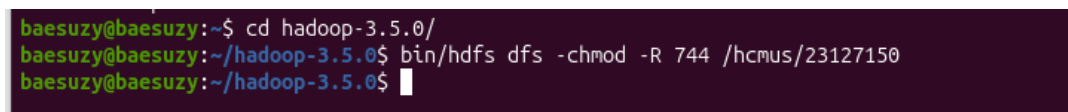
Hình 33: Kết quả chạy `hadoop-test.jar` lần đầu.

Ngoài dòng WARN ra thì có một lỗi xuất hiện là file `/hcmus/23127150/baesuzy.txt` không có permission là 744. Lí do là lúc trước em đã dùng user `baesuzy` để upload file này và sau đó chỉ `chmod 744` cho folder `/hcmus/23127150` mà không flag `-R` để `chmod` cho tất cả các file bên trong folder.

Giờ em sẽ fix bằng cách `chmod 744` cho tất cả các file bên trong folder `/hcmus/23127150` bằng lệnh:

```
1 bin/hdfs dfs -chmod -R 744 /hcmus/23127150
```

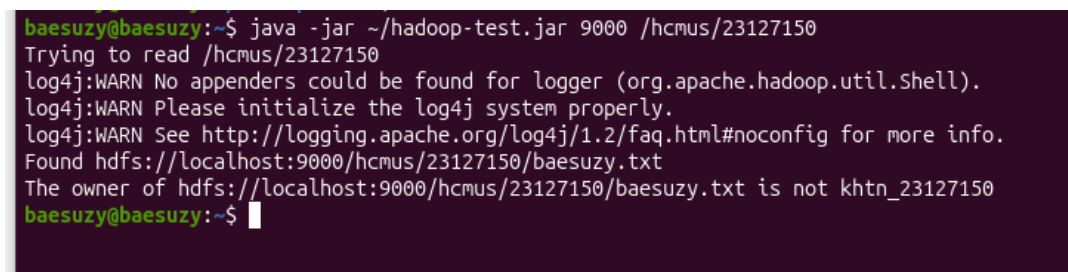
Kết quả như Hình 34.



```
baesuzy@baesuzy:~$ cd hadoop-3.5.0/
baesuzy@baesuzy:~/hadoop-3.5.0$ bin/hdfs dfs -chmod -R 744 /hcmus/23127150
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 34: `chmod 744` đệ quy cho `/hcmus/23127150`.

Kết quả sau khi chạy lại lệnh `java -jar ~/hadoop-test.jar 9000 /hcmus/23127150` như Hình 35.



```
baesuzy@baesuzy:~$ java -jar ~/hadoop-test.jar 9000 /hcmus/23127150
Trying to read /hcmus/23127150
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23127150/baesuzy.txt
The owner of hdfs://localhost:9000/hcmus/23127150/baesuzy.txt is not khtn_23127150
baesuzy@baesuzy:~$
```

Hình 35: Kết quả chạy JAR verify lần thứ hai.

Vẫn còn lỗi xuất hiện cho thấy rằng owner của `baesuzy.txt` không phải là `khtn_23127150`. Lí do là lúc trước em chỉ `chown` cho folder `/hcmus/23127150` mà không flag `-R` để `chown` cho tất cả các file bên trong folder.

```
1 bin/hdfs dfs -chown -R khtn_23127150 /hcmus/23127150
```

Kết quả như Hình 36.

```
baesuzy@baesuzy:~$ cd hadoop-3.5.0/
baesuzy@baesuzy:~/hadoop-3.5.0$ bin/hdfs dfs -chown -R khtn_23127150 /hcmus/23127150
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 36: `chown` đệ quy owner `khtn_23127150` cho `/hcmus/23127150`.

Kết quả sau khi chạy lại script như Hình 37.

```
baesuzy@baesuzy:~$ java -jar ~/hadoop-test.jar 9000 /hcmus/23127150
Trying to read /hcmus/23127150
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23127150/baesuzy.txt
Your student ID: 23127150 (ensure it matches your student ID)
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
File written at /home/baesuzy/23127150_verification.txt
baesuzy@baesuzy:~$
```

Hình 37: Kết quả chạy JAR verify lần thứ ba (thành công).

Lúc này thì đã chạy ra file `23127150_verification.txt` thành công nhưng lại xuất hiện thông báo: `The first method to get MAC address is failed: Could not get network interface.` và `Trying the alternative method.` Theo em suy đoán thì lỗi này có thể do sử dụng máy ảo nên việc lấy MAC address của interface gặp khó khăn. Tuy nhiên, script vẫn chạy thành công và tạo ra file `23127150_verification.txt` nên em nghĩ là không ảnh hưởng gì tới kết quả cuối cùng. Giờ em sẽ kiểm tra file `23127150_verification.txt` xem nội dung bên trong bằng `cat ~/23127150_verification` (Hình 38).

```
baesuzy@baesuzy:~$ cat 23127150_verification.txt
MAC=00-0C-29-47-7D-1B
fe7db8e55761cd39c3010c89f7e31322b1da1f7142296db68e2027914801e942baesuzy@baesuzy:~$
```

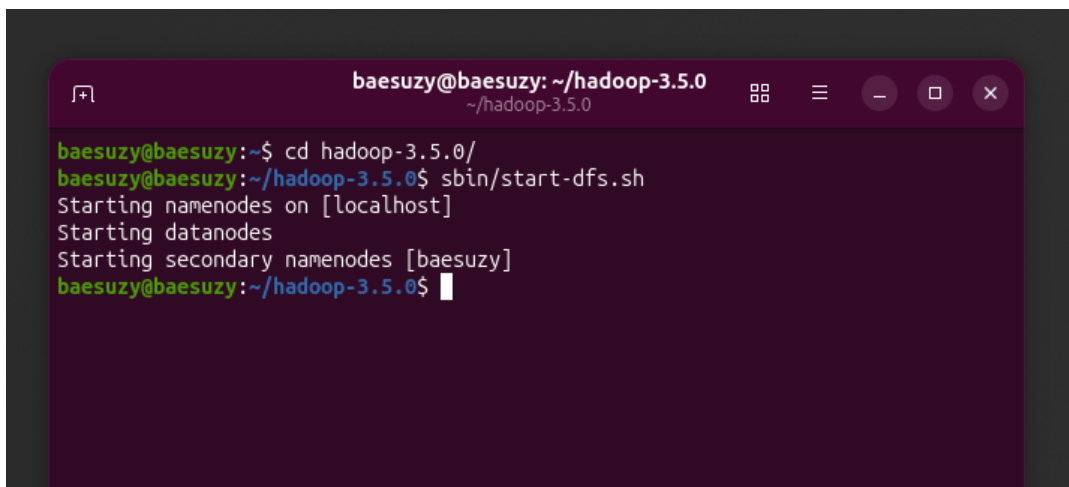
Hình 38: Nội dung file `23127150_verification.txt`.

Như vậy là phần cá nhân của em đã hoàn tất.

3 Một số lỗi xảy ra trong quá trình

3.1 HDFS không start thành công sau khi reboot máy

Sau khi reboot máy ảo Ubuntu, em thử `sbin/start-dfs.sh` thì hiện log như Hình 39, tuy nhiên vào `localhost:9870` thì không được.



```

baesuzu@baesuzu: ~/hadoop-3.5.0
baesuzu@baesuzu:~$ cd hadoop-3.5.0/
baesuzu@baesuzu:~/hadoop-3.5.0$ sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [baesuzu]
baesuzu@baesuzu:~/hadoop-3.5.0$

```

Hình 39: Log `start-dfs.sh` sau khi reboot.

Điều này chứng tỏ HDFS đã gặp vấn đề gì đó sau khi reboot. Em có thử dùng Claude để tra cứu về vấn đề như Hình 40.

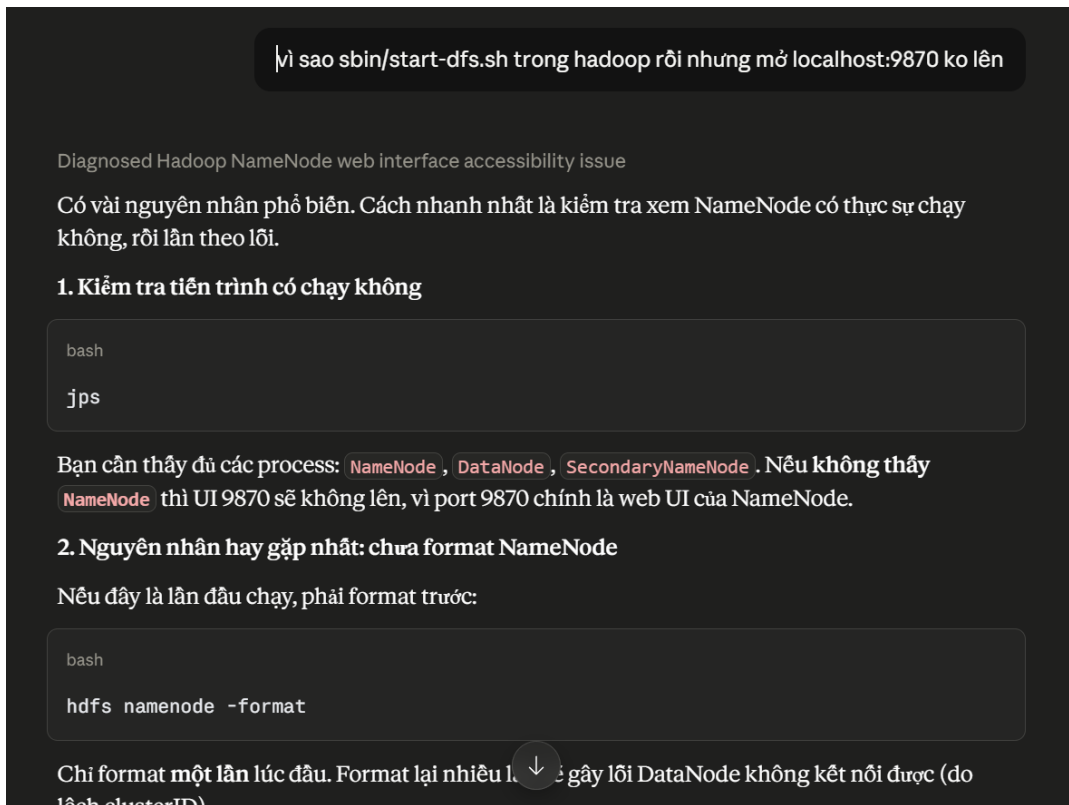
Khi dùng `jps` như hướng dẫn thì đúng là không thấy NameNode. Vì vậy, em sẽ thử chạy format lại HDFS bằng lệnh `bin/hdfs namenode -format`. Tất nhiên là khi format như vậy thì toàn bộ dữ liệu trước đó trên HDFS sẽ bị mất. Đây là sau khi format lại (Hình 41).

Lúc này, em sẽ stop HDFS bằng lệnh `sbin/stop-dfs.sh` và start lại bằng lệnh `sbin/start-dfs.sh`. Sau khi start lại thì vào `localhost:9870` thì đã được (Hình 42 và Hình 43).

Như vậy là em đã xác nhận được nguyên nhân nhưng giờ em sẽ thử khắc phục nó để những lần sau không phải format lại nữa. Tiếp tục dùng Claude để tìm cách fix (Hình 44).

Kiểm tra thư mục `/tmp` thử xem có thật sự như Claude nói hay không thì thấy đúng thật là có các data liên quan tới HDFS nên em đoán là Claude nói có thể đúng (Hình 45). Nhưng để chắc chắn thì em sẽ follow theo hướng dẫn của nó là config cho HDFS lưu ở một directory khác thay vì `/tmp`.

1. Tạo 2 thư mục để lưu lâu dài (Hình 46):
2. Chỉnh sửa file `etc/hadoop/hdfs-site.xml` để thay đổi đường dẫn lưu trữ của HDFS với đoạn cấu hình sau:



Hình 40: Tra cứu vấn đề bằng Claude.

```

1 <property>
2   <name>dfs.namenode.name.dir</name>
3   <value>file:///home/baesuzy/hadoop_data/namenode</value>
4 </property>
5 <property>
6   <name>dfs.datanode.data.dir</name>
7   <value>file:///home/baesuzy/hadoop_data/datanode</value>
8 </property>

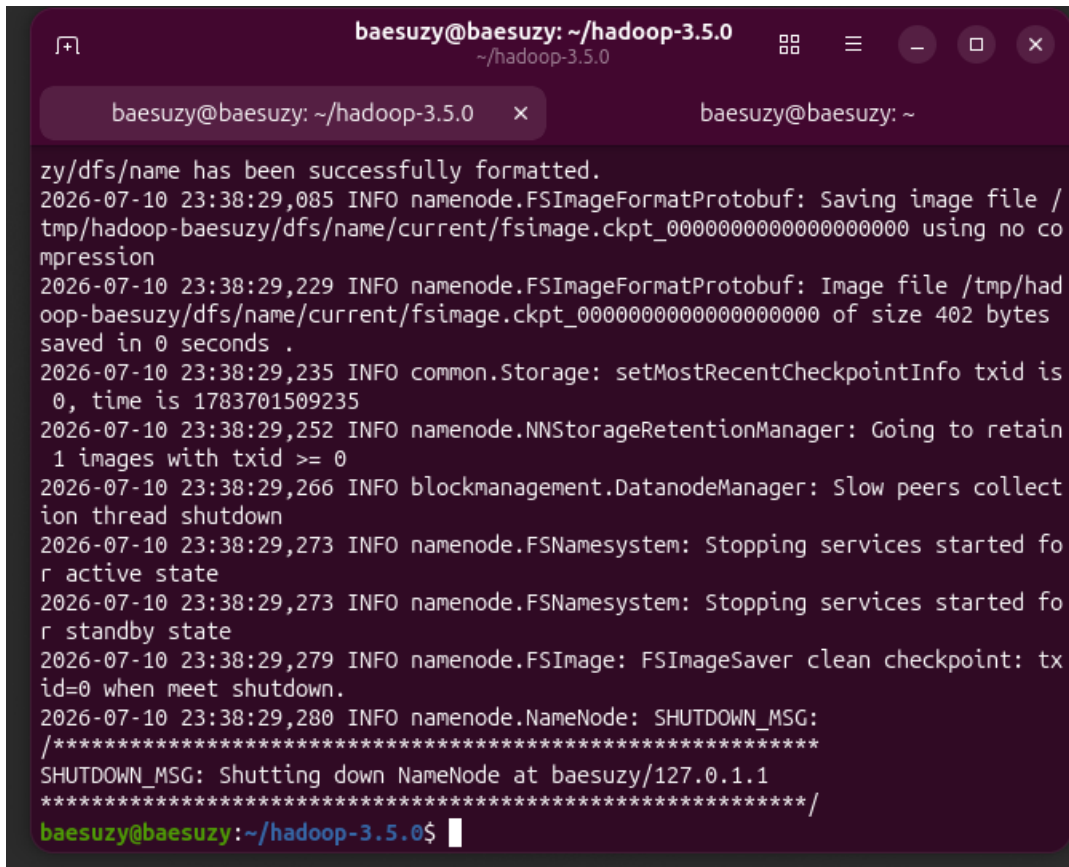
```

Kết quả chỉnh sửa như Hình 47.

3. Stop HDFS → Format lại HDFS → Start HDFS và thêm một file vào HDFS để kiểm tra xem có bị mất dữ liệu hay không (Hình 48).

4. Sau khi reboot máy ảo Ubuntu thì em kiểm tra thấy HDFS start được mà không gặp lỗi như vừa rồi và check file `test.txt` put trước đó vẫn còn (Hình 49).

Như vậy là lỗi này được fix.

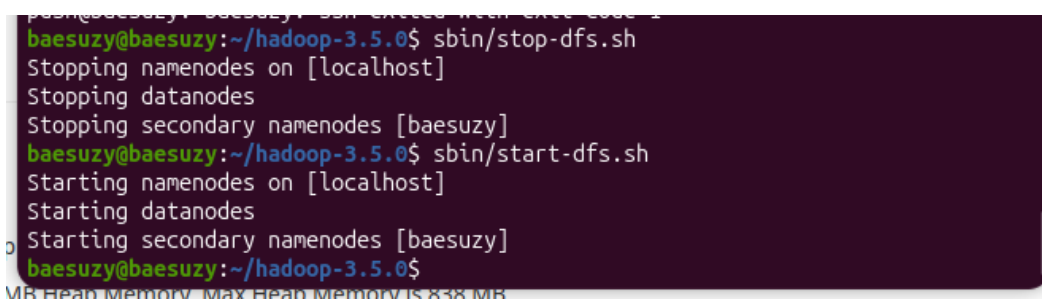


```

baesuzy@baesuzy: ~/hadoop-3.5.0
~/hadoop-3.5.0
baesuzy@baesuzy: ~/hadoop-3.5.0 x baesuzy@baesuzy: ~
zy/dfs/name has been successfully formatted.
2026-07-10 23:38:29,085 INFO namenode.FSImageFormatProtobuf: Saving image file /
tmp/hadoop-baesuzy/dfs/name/current/fsimage.ckpt_000000000000000000 using no co
mpression
2026-07-10 23:38:29,229 INFO namenode.FSImageFormatProtobuf: Image file /tmp/had
oop-baesuzy/dfs/name/current/fsimage.ckpt_000000000000000000 of size 402 bytes
saved in 0 seconds .
2026-07-10 23:38:29,235 INFO common.Storage: setMostRecentCheckpointInfo txid is
0, time is 1783701509235
2026-07-10 23:38:29,252 INFO namenode.NNStorageRetentionManager: Going to retain
1 images with txid >= 0
2026-07-10 23:38:29,266 INFO blockmanagement.DatanodeManager: Slow peers collect
ion thread shutdown
2026-07-10 23:38:29,273 INFO namenode.FSNamesystem: Stopping services started fo
r active state
2026-07-10 23:38:29,273 INFO namenode.FSNamesystem: Stopping services started fo
r standby state
2026-07-10 23:38:29,279 INFO namenode.FSImage: FSImageSaver clean checkpoint: tx
id=0 when meet shutdown.
2026-07-10 23:38:29,280 INFO namenode.NameNode: SHUTDOWN_MSG:
/*****
SHUTDOWN_MSG: Shutting down NameNode at baesuzy/127.0.1.1
*****/
baesuzy@baesuzy:~/hadoop-3.5.0$

```

Hình 41: HDFS sau khi format lại.

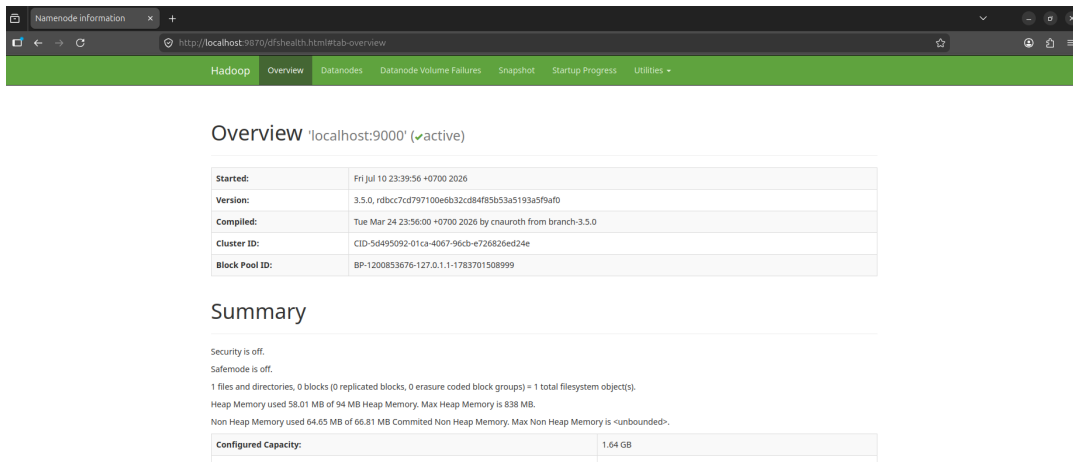


```

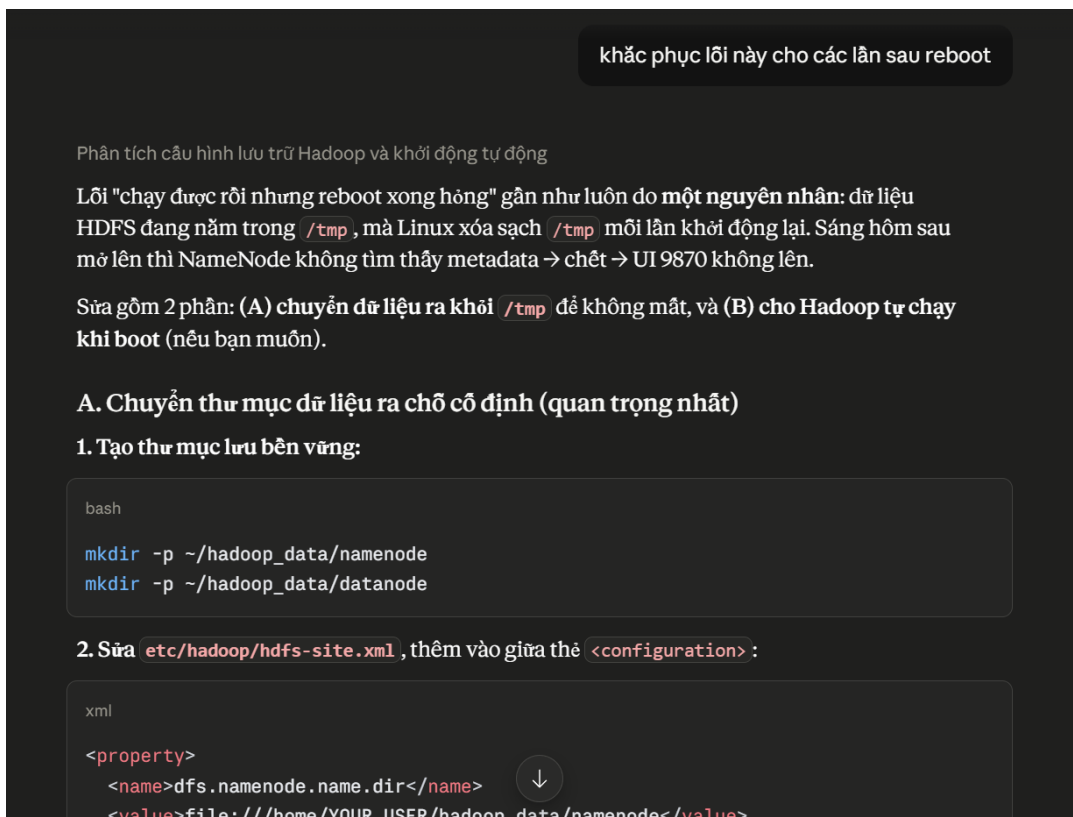
baesuzy@baesuzy:~/hadoop-3.5.0$ sbin/stop-dfs.sh
Stopping namenodes on [localhost]
Stopping datanodes
Stopping secondary namenodes [baesuzy]
baesuzy@baesuzy:~/hadoop-3.5.0$ sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [baesuzy]
baesuzy@baesuzy:~/hadoop-3.5.0$

```

Hình 42: HDFS hoạt động lại sau khi start lại (1).



Hình 43: HDFS hoạt động lại sau khi start lại (2).



Hình 44: Tìm cách khắc phục bằng Claude.


```
baesuzy@baesuzy:~/hadoop-3.5.0$ ls /tmp
VMwareDnD
hadoop-baesuzy
hadoop-baesuzy-datanode.pid
hadoop-baesuzy-namenode.pid
hadoop-baesuzy-secondarynamenode.pid
hsperfdata_baesuzy
jetty-0_0_0_0-9868-secondary-_-any-15259598867100953585
jetty-0_0_0_0-9870-hdfs-_-any-7012577916384444889
jetty-127_0_0_1-33981-datanode-_-any-10256952803990287071
snap-private-tmp
systemd-private-28a52af662ac451eb35f27ff056baaf4-ModemManager.service-xFRxcb
systemd-private-28a52af662ac451eb35f27ff056baaf4-chrony.service-Hm0yjc
systemd-private-28a52af662ac451eb35f27ff056baaf4-colord.service-JVzBiv
systemd-private-28a52af662ac451eb35f27ff056baaf4-geoclue.service-MCGlo4
systemd-private-28a52af662ac451eb35f27ff056baaf4-polkit.service-dyPFcc
systemd-private-28a52af662ac451eb35f27ff056baaf4-power-profiles-daemon.service-0
N9b9x
systemd-private-28a52af662ac451eb35f27ff056baaf4-switcheroo-control.service-IEcT
sU
systemd-private-28a52af662ac451eb35f27ff056baaf4-systemd-logind.service-pzeafS
systemd-private-28a52af662ac451eb35f27ff056baaf4-upower.service-sYgTx7
vmware-root_1047-4248615092
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 45: Dữ liệu HDFS nằm trong /tmp.

```
baesuzy@baesuzy:~/hadoop-3.5.0$ mkdir -p ~/hadoop-data/namenode
baesuzy@baesuzy:~/hadoop-3.5.0$ mkdir -p ~/hadoop-data/datanode
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 46: Tạo 2 thư mục lưu trữ lâu dài cho HDFS.

```

baesuzy@baesuzy: ~/hadoop-3.5.0 — nano etc/hadoo...
~ /hadoop-3.5.0
GNU nano 8.7.1      etc/hadoop/hdfs-site.xml
-->
<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/baesuzy/hadoop_data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/baesuzy/hadoop_data/datanode</value>
  </property>
</configuration>

[ Wrote 32 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
0 rep of 94 MB Heap Memory. Max Heap Memory is 838 MB.
MB of 66.81 MB Committed Mem Heap Memory. Max Mem Heap Memory is 838 MB.

```

Hình 47: Cấu hình name.dir và data.dir trong hdfs-site.xml.

```

baesuzy@baesuzy:~/hadoop-3.5.0$ bin/hdfs dfs -put test.txt /test.txt
baesuzy@baesuzy:~/hadoop-3.5.0$ bin/hdfs dfs -ls /
Found 1 items
-rw-r--r--  1 baesuzy supergroup      8 2026-07-11 00:03 /test.txt
baesuzy@baesuzy:~/hadoop-3.5.0$
MB Heap Memory. Max Heap Memory is 838 MB.

```

Hình 48: Stop, format lại và start HDFS với thư mục lưu trữ mới.

```

baesuzy@baesuzy: ~/hadoop-3.5.0
~ /hadoop-3.5.0
baesuzy@baesuzy:~$ cd hadoop
bash: cd: hadoop: No such file or directory
baesuzy@baesuzy:~$ cd hadoop
hadoop-3.5.0/ hadoop-data/ hadoop_data/
baesuzy@baesuzy:~$ cd hadoop
hadoop-3.5.0/ hadoop-data/ hadoop_data/
baesuzy@baesuzy:~$ cd hadoop
hadoop-3.5.0/ hadoop-data/ hadoop_data/
baesuzy@baesuzy:~$ cd hadoop-3.5.0/
baesuzy@baesuzy:~/hadoop-3.5.0$ sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [baesuzy]
baesuzy@baesuzy:~/hadoop-3.5.0$ bin/hdfs dfs -ls /
Found 1 items
-rw-r--r-- 1 baesuzy supergroup      8 2026-07-11 00:03 /test.txt
baesuzy@baesuzy:~/hadoop-3.5.0$

```

Hình 49: Sau reboot: HDFS start bình thường và dữ liệu vẫn còn.

4 Warm up với WordCount

Phần Setup mọi thứ cho giải bài toán WordCount bằng Hadoop MapReduce của em sẽ follow theo hướng dẫn Getting Started with Scala and sbt on the command line [7].

4.1 Cài đặt sbt trên Ubuntu

Trong hướng dẫn trên, để cài đặt sbt trên Ubuntu thì có một đường dẫn tới một hướng dẫn khác: Installing sbt on Linux [6].

Trong đây có hướng dẫn trên Ubuntu chạy bash như sau:

```

1 sudo apt-get update
2 sudo apt-get install apt-transport-https curl gnupg -yqq
3 echo "deb https://repo.scala-sbt.org/scalasbt/debian all main" | sudo tee /etc/
  apt/sources.list.d/sbt.list
4 echo "deb https://repo.scala-sbt.org/scalasbt/debian /" | sudo tee /etc/apt/
  sources.list.d/sbt_old.list
5 curl -sL "https://keyserver.ubuntu.com/pks/lookup?op=get&search=0
  x2EE0EA64E40A89B84B2DF73499E82A75642AC823" | sudo -H gpg --no-default-keyring
  --keyring gnupg-ring:/etc/apt/trusted.gpg.d/scalasbt-release.gpg --import
6 sudo chmod 644 /etc/apt/trusted.gpg.d/scalasbt-release.gpg
7 sudo apt-get update

```

```
8 sudo apt-get install sbt
```

Kết quả như Hình 50.

```
baesuzy@baesuzy: ~/hadoop-3.5.0
0 upgraded, 1 newly installed, 0 to remove and 15 not upgraded.
Need to get 21.9 kB of archives.
After this operation, 55.3 kB of additional disk space will be used.
Get:1 https://scala.jfrog.io/artifactory/debian all/main amd64 sbt all 2.0.1 [21.9 kB]
Fetched 21.9 kB in 2s (13.0 kB/s)
Selecting previously unselected package sbt.
(Reading database... 156683 files and directories currently installed.)
Preparing to unpack .../apt/archives/sbt_2.0.1_all.deb...
Unpacking sbt (2.0.1)...
Setting up sbt (2.0.1)...
Creating system group: sbt
Creating system user: sbt in sbt with sbt daemon-user and shell /bin/false
Processing triggers for man-db (2.13.1-1build1)...
baesuzy@baesuzy:~/hadoop-3.5.0$ sbt
[info] server was not detected. starting an instance
[error] Neither build.sbt nor a 'project' directory in the current directory: /home/baesuzy/hadoop-3.5.0
[error] run 'sbt new', touch build.sbt, or run 'sbt --allow-empty'.
[error]
[error] To opt out of this check, create /home/baesuzy/.config/sbt/sbtopts with:
[error] --allow-empty
[error] failed to connect to server
baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 50: Cài đặt sbt trên Ubuntu.

Với lệnh trên thì em cài đặt thành công sbt với kiểm tra thử thì thấy đã gọi trên terminal được. Lỗi hiển thị trên hình không phải vấn đề vì đây chỉ để test xem đã cài đặt sbt chưa.

4.2 Tạo project Scala với sbt

sbt cung cấp cho chúng ta một câu lệnh để tự động khởi tạo 1 project Scala, giờ em sẽ ở ~ (home của em) và tạo project Scala với tên wordcount bằng lệnh:

```
1 sbt new scala/scala3.g8
```

Kết quả như Hình 51.

Project tạo ra có cấu trúc như Hình 52.

Trong đó:

- **build.sbt**: là file build chính của project. File này khai báo tên project, phiên bản, phiên bản Scala sử dụng và các thư viện (dependencies) mà project cần. Khi chạy sbt, công cụ này sẽ đọc file build.sbt để biết cách build project.

```
baesuzy@baesuzy:~$ sbt new scala/scala3.g8
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
A template to demonstrate a minimal Scala 3 application

name [Scala 3 Project Template]: wordcount

Template applied in /home/baesuzy/./wordcount

baesuzy@baesuzy:~$
```

Hình 51: Tạo project Scala bằng `sbt new`.

```
baesuzy@baesuzy:~/wordcount$ tree .
.
├── build.sbt
├── project
│   └── build.properties
├── README.md
├── src
│   ├── main
│   │   └── scala
│   │       └── Main.scala
│   └── test
│       └── scala
│           └── MySuite.scala
└── 7 directories, 5 files

baesuzy@baesuzy:~/wordcount$
```

Hình 52: Cấu trúc project Scala wordcount.

- `project/`: là thư mục chứa các cấu hình bổ sung cho chính `sbt`. Ví dụ file `project/build.properties` chỉ định phiên bản `sbt` được dùng cho project. Sau này khi cần thêm plugin (như `sbt-assembly`), em cũng sẽ khai báo trong thư mục này qua file `project/plugins.sbt`.
- `src/main/scala/`: là nơi chứa code Scala chính của chương trình. Đây là thư mục mà em sẽ đặt code `WordCount` vào. Mặc định template tạo sẵn một file `Main.scala` ở đây.
- `src/test/scala/`: là nơi chứa code test cho project. Template tạo sẵn file `MySuite.scala` để làm ví dụ. Vì bài này em không viết test nên sẽ không đụng tới thư mục này.
- `README.md`: là file mô tả project, do template tạo sẵn.

Như vậy tiếp theo phần code Scala cho `wordcount` sẽ được viết trong thư mục `src/main/scala`.

4.3 Viết code Scala cho WordCount

4.3.1 WordCount.scala

File `WordCount.scala` là file driver, tức là nơi định nghĩa và cấu hình cho một job MapReduce rồi submit nó lên Hadoop. Code của em như sau:

```

1 package WordCount
2
3 import org.apache.hadoop.conf.Configuration
4 import org.apache.hadoop.fs.Path
5 import org.apache.hadoop.io.{IntWritable, Text}
6 import org.apache.hadoop.mapreduce.Job
7 import org.apache.hadoop.mapreduce.lib.input.{FileInputFormat, TextInputFormat}
8 import org.apache.hadoop.mapreduce.lib.output.{FileOutputFormat,
9     TextOutputFormat}
10
11 object WordCountJob {
12     def main(args: Array[String]): Unit = {
13         if (args.length != 2) {
14             println("usage: [input] [output]")
15             System.exit(-1)
16         }
17
18         val job = Job.getInstance(new Configuration())
19         job.setJarByClass(classOf[WordCountJob.type])
20         job.setMapperClass(classOf[WordCountMapper])
21         job.setReducerClass(classOf[WordCountReducer])
22         job.setCombinerClass(classOf[WordCountReducer])
23
24         job.setInputFormatClass(classOf[TextInputFormat])
25         job.setOutputFormatClass(classOf[TextOutputFormat[Text, IntWritable]])

```

```

25     job.setOutputKeyClass(classOf[Text])
26     job.setOutputValueClass(classOf[IntWritable])
27
28     FileInputFormat.setInputPaths(job, new Path(args(0)))
29     FileOutputFormat.setOutputPath(job, new Path(args(1)))
30
31     job.submit()
32     System.exit(if (job.waitForCompletion(true)) 0 else 1)
33 }
34 }

```

Em giải thích từng thành phần như sau:

- Phần `import`: em import các class của Hadoop cần dùng. `Configuration` là cấu hình chung của Hadoop, `Path` để chỉ đường dẫn input/output trên HDFS, `IntWritable` và `Text` là các kiểu dữ liệu mà Hadoop dùng để truyền qua mạng (thay cho `Int` và `String` thông thường của Java/Scala), `Job` là class đại diện cho một job MapReduce, còn `FileInputFormat`, `TextInputFormat`, `FileOutputFormat`, `TextOutputFormat` là các class quy định cách đọc input và ghi output.
- `object WordCountJob` với hàm `main`: đây là điểm bắt đầu chạy của chương trình. Vì đây là chương trình chạy độc lập nên em cần một hàm `main`.
- `if (args.length != 2)`: em kiểm tra tham số đầu vào. Chương trình cần đúng 2 tham số là đường dẫn input và đường dẫn output, nếu không đủ thì in ra hướng dẫn dùng và thoát.
- `val job = Job.getInstance(new Configuration())`: tạo ra một job MapReduce mới với cấu hình mặc định của Hadoop.
- `job.setJarByClass(...)`: chỉ cho Hadoop biết class nào nằm trong file JAR để nó tìm ra file JAR chứa code job và phân phối tới các node chạy.
- `job.setMapperClass(classOf[WordCountMapper])` và `job.setReducerClass(classOf[WordCountReducer])`: khai báo class Mapper và Reducer mà job sẽ sử dụng, tương ứng với 2 file `Mapper.scala` và `Reducer.scala`.
- `job.setCombinerClass(classOf[WordCountReducer])`: em dùng luôn Reducer làm Combiner. Combiner giống như một bước “reduce trước” chạy ngay tại mỗi máy map để cộng dồn cục bộ trước khi gửi qua mạng, nhờ vậy giảm được lượng dữ liệu shuffle. Vì phép cộng của WordCount có tính kết hợp và giao hoán nên dùng Reducer làm Combiner là an toàn.
- `job.setInputFormatClass(...)` và `job.setOutputFormatClass(...)`: quy định định dạng đọc input là `TextInputFormat` (đọc file text theo từng dòng) và ghi output là `TextOutputFormat` (ghi ra dạng text `key<tab>value`).

- `job.setOutputKeyClass(classOf[Text])` và `job.setOutputValueClass(classOf[IntWritable])`: khai báo kiểu dữ liệu của key và value ở output là `Text` (chữ) và `IntWritable` (số nguyên), khớp với những gì Reducer ghi ra.
- `FileInputFormat.setInputPaths(...)` và `FileOutputFormat.setOutputPath(...)`: gán đường dẫn input (`args(0)`) và output (`args(1)`) cho job. Đường dẫn output phải là thư mục chưa tồn tại, nếu đã tồn tại thì Hadoop sẽ báo lỗi.
- `job.submit()` và `job.waitForCompletion(true)`: submit job lên Hadoop rồi chờ cho tới khi job chạy xong. `waitForCompletion(true)` trả về `true` nếu job thành công, và em dùng kết quả này để trả về exit code tương ứng.

4.3.2 Mapper.scala

File `Mapper.scala` định nghĩa phần Map của job. Nhiệm vụ của Mapper là đọc từng dòng input rồi phát ra (emit) các cặp key-value trung gian. Code của em như sau:

```

1 package WordCount
2
3 import java.io.IOException
4 import java.util.StringTokenizer
5
6 import org.apache.hadoop.io.{IntWritable, Text}
7 import org.apache.hadoop.mapreduce.Mapper
8
9 class WordCountMapper extends Mapper[Object, Text, Text, IntWritable] {
10   val one = new IntWritable(1)
11   val word = new Text()
12
13   override def map(key: Object, value: Text, context: Mapper[Object, Text, Text,
14     IntWritable]#Context)
15     : Unit = {
16
17     val itr = new StringTokenizer(value.toString)
18
19     while (itr.hasMoreTokens) {
20       val token = itr.nextToken().toLowerCase
21
22       if (token.length > 0) {
23         val c = token.charAt(0)
24
25         if ("fithcmus".contains(c)) {
26           word.set(c.toString)
27           context.write(word, one)
28         }
29       }
30     }
31   }
32 }

```



```

29     }
30   }
31 }

```

Em giải thích từng thành phần như sau:

- `class WordCountMapper extends Mapper[Object, Text, Text, IntWritable]`: em kế thừa class `Mapper` của Hadoop. Bốn kiểu trong dấu ngoặc lần lượt là kiểu của key input, value input, key output, value output. Ở đây key input là `Object` (offset của dòng trong file, em không dùng tới), value input là `Text` (nội dung một dòng), còn key output là `Text` và value output là `IntWritable`.
- `val one = new IntWritable(1)` và `val word = new Text()`: em khai báo sẵn hai biến dùng lại nhiều lần để khỏi phải tạo object mới liên tục trong vòng lặp, giúp tiết kiệm bộ nhớ. `one` luôn mang giá trị 1, còn `word` là biến để chứa key sẽ emit.
- `override def map(...)`: đây là hàm được Hadoop gọi cho mỗi dòng input. Tham số value chính là nội dung của dòng đang xử lý, còn `context` là nơi để em ghi kết quả trung gian ra.
- `val itr = new StringTokenizer(value.toString)`: em tách dòng thành từng token (từ) theo khoảng trắng.
- `val token = itr.nextToken().toLowerCase`: với mỗi từ, em chuyển về chữ thường để không phân biệt hoa thường (ví dụ HCMUS và hcmus được xem là như nhau).
- `val c = token.charAt(0)`: em lấy ký tự đầu tiên của từ.
- `if ("fithcmus".contains(c))`: đây là phần logic riêng của bài. Em chỉ đếm những từ có ký tự đầu nằm trong tập {f, i, t, h, c, m, u, s} (chính là các chữ cái của FIT HCMUS). Những từ bắt đầu bằng ký tự khác sẽ bị bỏ qua.
- `word.set(c.toString)` và `context.write(word, one)`: nếu thỏa điều kiện, em đặt key là ký tự đầu đó rồi emit cặp (ký tự, 1) ra ngoài. Sau bước Map, mỗi lần một từ hợp lệ xuất hiện sẽ tạo ra một cặp như vậy để Reducer cộng lại sau.

4.3.3 Reducer.scala

File `Reducer.scala` định nghĩa phần Reduce của job. Sau bước Map và shuffle, Hadoop gom tất cả các value có cùng key lại thành một nhóm rồi gọi Reducer một lần cho mỗi key. Nhiệm vụ của Reducer ở đây là cộng tất cả các số 1 lại để ra tổng số lần xuất hiện. Code của em như sau:

```

1 package WordCount
2
3 import org.apache.hadoop.io.{IntWritable, Text}
4 import org.apache.hadoop.mapreduce.Reducer

```

```

5
6 class WordCountReducer extends Reducer[Text, IntWritable, Text, IntWritable] {
7   val result = new IntWritable()
8
9   override def reduce(key: Text, values: java.lang.Iterable[IntWritable],
10                        context: Reducer[Text, IntWritable, Text, IntWritable]#
11                        Context): Unit = {
12     var sum = 0
13     val iter = values.iterator()
14
15     while (iter.hasNext) {
16       sum += iter.next().get()
17     }
18
19     result.set(sum)
20     context.write(key, result)
21 }

```

Em giải thích từng thành phần như sau:

- `class WordCountReducer extends Reducer[Text, IntWritable, Text, IntWritable]`: em kế thừa class `Reducer` của Hadoop. Bốn kiểu trong ngoặc là kiểu key input, value input, key output, value output. Key và value input ở đây (`Text, IntWritable`) phải khớp với key và value output của Mapper.
- `val result = new IntWritable()`: em khai báo sẵn một biến để chứa kết quả tổng, cũng để dùng lại tránh tạo object mới mỗi lần.
- `override def reduce(key, values, context)`: hàm này được Hadoop gọi một lần cho mỗi key. Tham số `values` là danh sách tất cả các value ứng với key đó (ở đây là một loạt số 1).
- Vòng `while (iter.hasNext) sum += iter.next().get()`: em duyệt qua toàn bộ các value và cộng dồn lại. `.get()` dùng để lấy ra giá trị `Int` bên trong `IntWritable`.
- `result.set(sum)` và `context.write(key, result)`: em đặt tổng vừa tính vào `result` rồi ghi cặp (ký tự, tổng số lần xuất hiện) ra output. Đây chính là kết quả cuối cùng của `WordCount`.

Vì `Reducer` chỉ đơn thuần cộng các giá trị lại nên em có thể dùng nó làm luôn `Combiner` như đã khai báo ở file driver.

4.3.4 Flow chạy của WordCount

Tổng hợp lại 3 file trên, luồng chạy của chương trình `WordCount` diễn ra như sau:

1. Hadoop đọc file input trong thư mục input trên HDFS, tách thành từng dòng và đưa mỗi dòng cho hàm `map` trong `Mapper.scala`.
2. Với mỗi dòng, Mapper tách ra từng từ, chuyển về chữ thường, lấy ký tự đầu tiên. Nếu ký tự đó nằm trong tập `fithcmus` thì emit cặp (ký tự, 1).
3. Combiner (chính là Reducer) chạy cục bộ ngay tại mỗi máy map để cộng dồn sơ bộ các cặp cùng key, giúp giảm lượng dữ liệu phải truyền qua mạng ở bước shuffle.
4. Hadoop thực hiện shuffle & sort: gom tất cả các cặp có cùng key từ các Mapper lại, sắp xếp theo key rồi đưa từng nhóm key về cho Reducer.
5. Với mỗi key, hàm `reduce` trong `Reducer.scala` cộng tất cả các số 1 lại để ra tổng số lần xuất hiện, rồi ghi cặp (ký tự, tổng) ra output.
6. Kết quả cuối được ghi thành file `part-r-00000` trong thư mục output trên HDFS.

4.4 Build JAR file và chạy với Hadoop

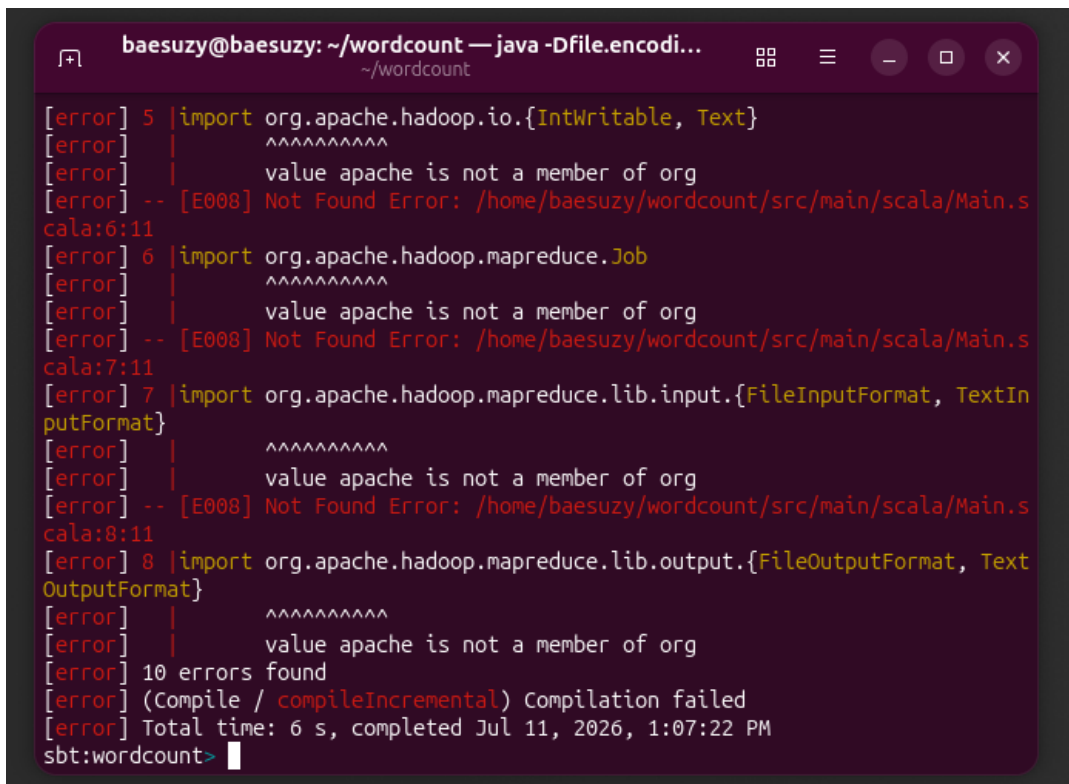
Sở dĩ cần build project Scala thành file JAR là vì Hadoop chạy code MapReduce dưới dạng file JAR. Khi em submit một job, Hadoop cần một gói duy nhất chứa toàn bộ các class đã biên dịch (Mapper, Reducer, driver) để có thể phân phối và chạy trên các node của cluster. Bản thân các file `.scala` là code nguồn, Hadoop không chạy trực tiếp được mà phải được compile thành bytecode rồi đóng gói lại thành `.jar`. Vì vậy em cần build project thành file JAR trước rồi mới đưa cho Hadoop chạy bằng lệnh `hadoop jar`.

Để build file JAR, trước hết phải đảm bảo mình đang trong project được tạo ra bằng `sbt`, ở đây là project `wordcount`. Tiếp theo, em sẽ dùng lệnh `sbt` để vào sbt shell. Tối đây theo hướng dẫn thì dùng lệnh `run` để chạy project Scala, tuy nhiên thì có lỗi do em chưa config dependencies của `hadoop` trong file `build.sbt` (Hình 53).

Em có đi tra cứu cách thêm dependencies của Hadoop vào file `build.sbt` thì thấy hướng dẫn ở file `build.sbt` mẫu của repo `salomvary/hadoop-mapreduce-tutorial` [5]. Cụ thể, em sẽ thêm sửa file `build.sbt` như Hình 54.

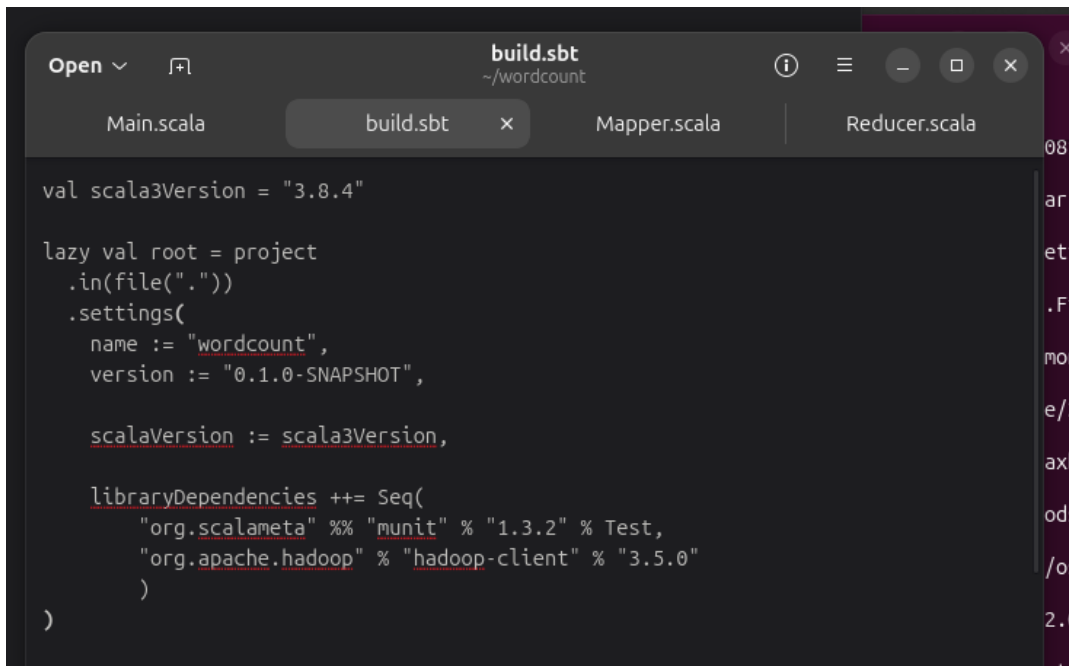
Dòng `"org.apache.hadoop" % "hadoop-client" % "3.5.0"` khai báo cho `sbt` biết project của em phụ thuộc vào thư viện `hadoop-client` phiên bản 3.5.0. Trong đó `org.apache.hadoop` là groupId (tổ chức phát hành thư viện), `hadoop-client` là artifactId (tên thư viện) và 3.5.0 là phiên bản. `hadoop-client` gói sẵn các API cần thiết để viết chương trình MapReduce như `Job`, `Mapper`, `Reducer`, `Text`, `IntWritable`... Nhờ khai báo dòng này, `sbt` sẽ tự động tải các thư viện Hadoop về và cho phép code của em `import` cũng như compile được. Em chọn đúng phiên bản 3.5.0 để khớp với phiên bản Hadoop đang cài trên máy.

Bây giờ, em sẽ chạy lại `sbt` và `run` (Hình 55).



```
baesuzy@baesuzy: ~/wordcount — java -Dfile.encodi...  
~/wordcount  
[error] 5 |import org.apache.hadoop.io.{IntWritable, Text}  
[error] |      ^^^^^^^^^  
[error] |      value apache is not a member of org  
[error] -- [E008] Not Found Error: /home/baesuzy/wordcount/src/main/scala/Main.s  
cala:6:11  
[error] 6 |import org.apache.hadoop.mapreduce.Job  
[error] |      ^^^^^^^^^  
[error] |      value apache is not a member of org  
[error] -- [E008] Not Found Error: /home/baesuzy/wordcount/src/main/scala/Main.s  
cala:7:11  
[error] 7 |import org.apache.hadoop.mapreduce.lib.input.{FileInputFormat, TextIn  
putFormat}  
[error] |      ^^^^^^^^^  
[error] |      value apache is not a member of org  
[error] -- [E008] Not Found Error: /home/baesuzy/wordcount/src/main/scala/Main.s  
cala:8:11  
[error] 8 |import org.apache.hadoop.mapreduce.lib.output.{FileOutputFormat, Text  
OutputFormat}  
[error] |      ^^^^^^^^^  
[error] |      value apache is not a member of org  
[error] 10 errors found  
[error] (Compile / compileIncremental) Compilation failed  
[error] Total time: 6 s, completed Jul 11, 2026, 1:07:22 PM  
sbt:wordcount>
```

Hình 53: Lỗi khi run do thiếu dependencies Hadoop.



```
Open ▾  build.sbt  
~/wordcount  
Main.scala  build.sbt  Mapper.scala  Reducer.scala  
val scala3Version = "3.8.4"  
  
lazy val root = project  
  .in(file("."))  
  .settings(  
    name := "wordcount",  
    version := "0.1.0-SNAPSHOT",  
  
    scalaVersion := scala3Version,  
  
    libraryDependencies ++= Seq(  
      "org.scalameta" %% "munit" % "1.3.2" % Test,  
      "org.apache.hadoop" % "hadoop-client" % "3.5.0"  
    )  
  )
```

Hình 54: Thêm dependency hadoop-client vào build.sbt.

```

100.0% [#####] 16.5 KiB (159.8 KiB / s)
https://repo1.maven.org/maven2/org/apache/avro/avro/1.11.5/avro-1.11.5.jar
100.0% [#####] 637.3 KiB (4.4 MiB / s)
https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-hdfs-client/3.5.0/hadoo...
100.0% [#####] 5.3 MiB (26.8 MiB / s)
https://repo1.maven.org/maven2/org/eclipse/jetty/jetty-util-ajax/9.4.58.v202508...
100.0% [#####] 65.6 KiB (486.2 KiB / s)
https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-auth/3.5.0/hadoop-auth-...
100.0% [#####] 112.1 KiB (1.1 MiB / s)
https://repo1.maven.org/maven2/io/netty/netty-transport-native-epoll/4.1.130.Fi...
100.0% [#####] 39.9 KiB (302.5 KiB / s)
https://repo1.maven.org/maven2/org/apache/kerby/kerby-pkix/2.0.3/kerby-pkix-2.0...
100.0% [#####] 195.9 KiB (1.6 MiB / s)
https://repo1.maven.org/maven2/org/glassfish/jersey/containers/jersey-container...
100.0% [#####] 74.0 KiB (556.1 KiB / s)
https://repo1.maven.org/maven2/org/glassfish/jersey/core/jersey-common/2.46/jer...
100.0% [#####] 1.2 MiB (4.9 MiB / s)
[info] Fetched artifacts of wordcount_3
[info] compiling 3 Scala sources to /home/baesuzy/wordcount/target/scala-3.8.4/c
lasses ...
[info] running WordCount.WordCountJob
usage: [input] [output]
[info] shutting down sbt server
baesuzy@baesuzy:~/wordcount$

```

Hình 55: Chạy lại `sbt run` sau khi thêm dependency.

Giờ thì đã compile được thành 1 file jar trong thư mục `target/scala-3.8.4/wordcount_3-0.1.0-SNAPSHOT.jar`. (Hình 56).

Giờ em tạo thử 1 thư mục `input` trên HDFS và upload 1 file text vào đó để test thử chương trình WordCount.

```

1 bin/hdfs dfs -mkdir /input
2 bin/hdfs dfs -put ~/Downloads/words.txt /input
3 bin/hadoop -jar ~/wordcount/target/scala-3.8.4/wordcount_3-0.1.0-SNAPSHOT.jar /
  input/words.txt /output

```

Kết quả như Hình 57.

Như vậy là vẫn chưa chạy được và gặp lỗi `NoClassDefFoundError`. Lí do là file JAR mà `sbt` build ra theo cách thông thường chỉ chứa code của riêng em (các class WordCount đã compile), chứ không đóng gói kèm các thư viện phụ thuộc như Scala library hay Hadoop client. Khi Hadoop chạy file JAR này, tới lúc cần nạp một class của Scala (ví dụ `scala/Predef`) hay của thư viện khác thì không tìm thấy trong JAR nên báo `NoClassDefFoundError`. Nói cách khác, JAR thường bị thiếu các class mà chương trình cần lúc runtime.

Cách fix lỗi này em sẽ build lại với assembly plugin của sbt. Cụ thể các bước:

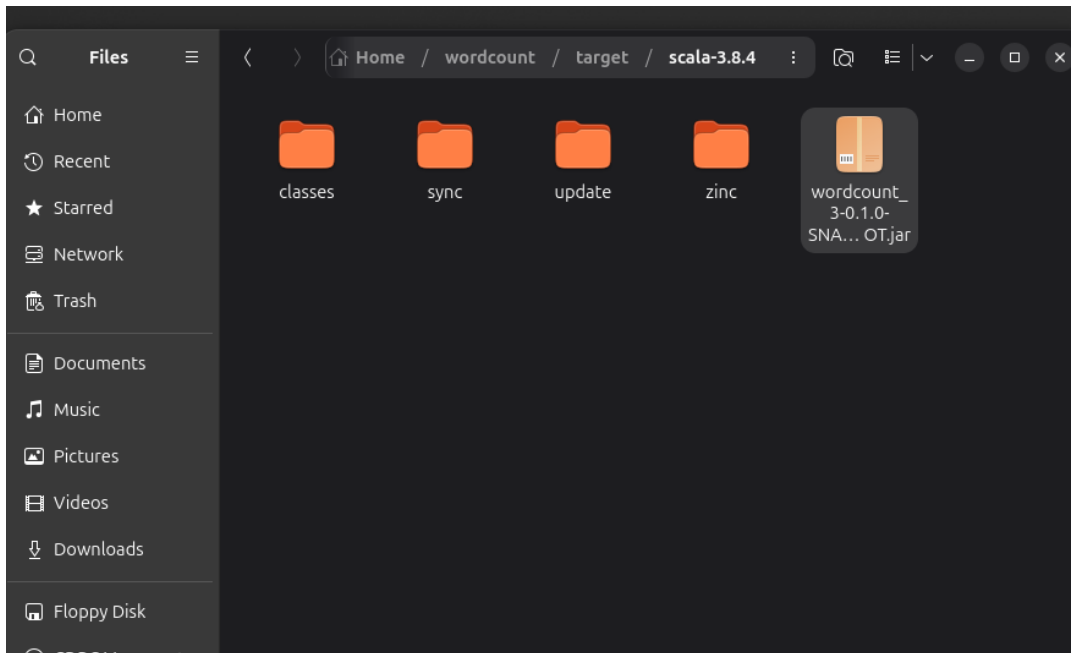
1. Tạo thêm plugin `project/plugins.sbt` với nội dung:

```

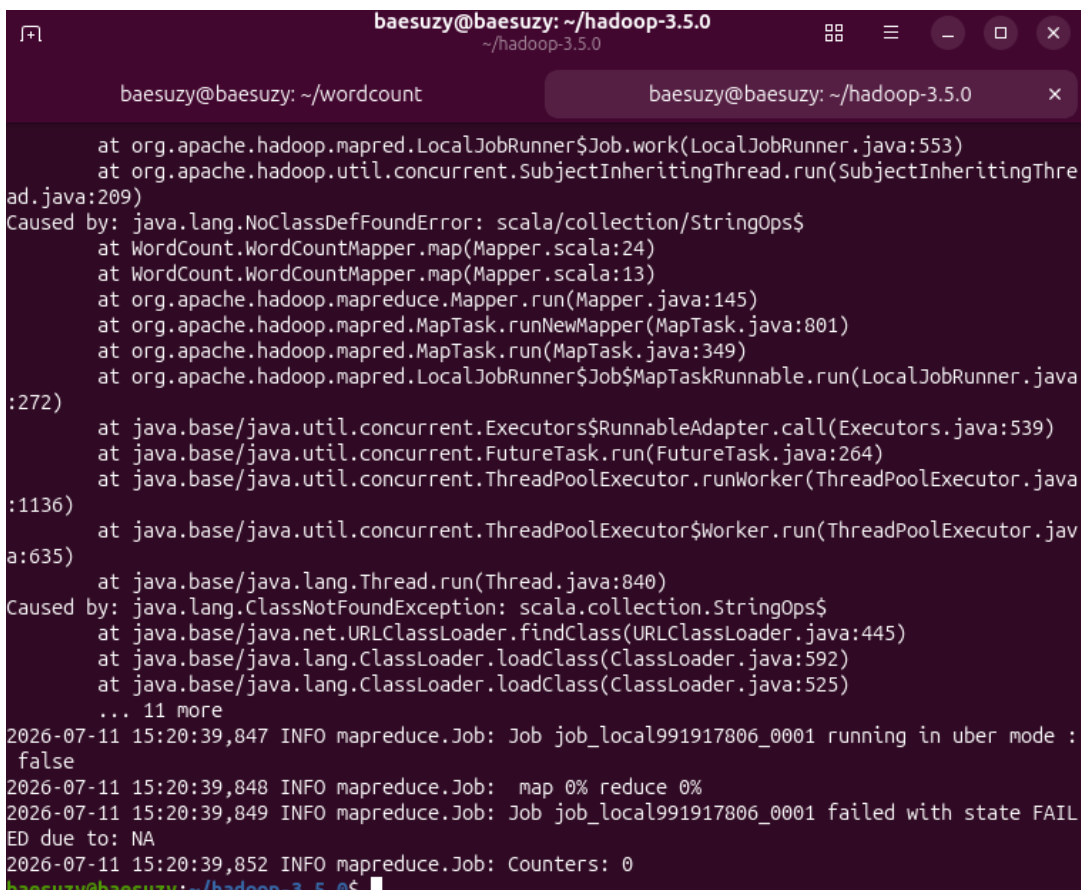
1 addSbtPlugin("com.eed3si9n" % "sbt-assembly" % "2.3.1")

```

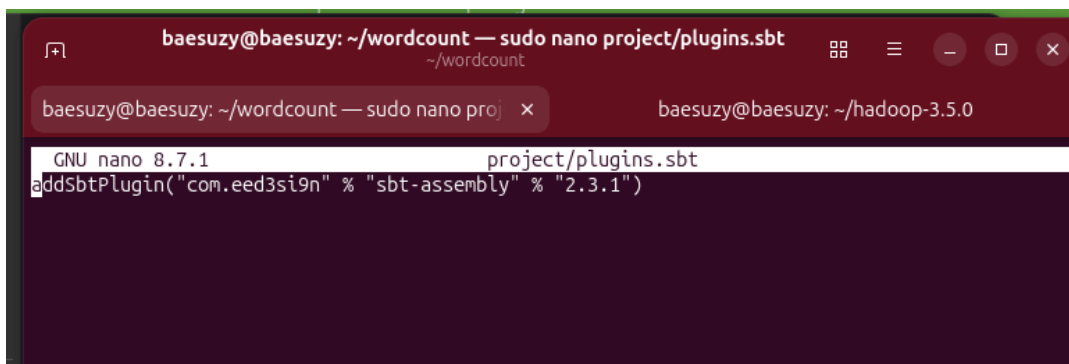
Kết quả như Hình 58.



Hình 56: File JAR được compile trong thư mục `target`.



Hình 57: Lỗi `NoClassDefFoundError` khi chạy JAR thường.



Hình 58: Khai báo plugin sbt-assembly trong project/plugins.sbt.

2. Modify file build.sbt:

```
1 assembly / assemblyMergeStrategy := {
2     case PathList("META-INF", xs @ _*) => MergeStrategy.discard
3     case x => MergeStrategy.first
4 }
```

Kết quả như Hình 59.

Đoạn code trên cấu hình cách plugin **sbt-assembly** xử lý khi có nhiều file trùng đường dẫn lúc gộp tất cả dependencies vào một fat JAR. Khi gộp nhiều thư viện lại, sẽ có những file bị trùng tên/trùng đường dẫn giữa các thư viện, và mặc định **sbt-assembly** sẽ báo lỗi conflict và dừng lại. Vì vậy em cần khai báo **assemblyMergeStrategy** để chỉ cho nó biết phải làm gì với từng loại file:

- **case PathList("META-INF", xs @ _*) => MergeStrategy.discard**: với các file nằm trong thư mục **META-INF** (thường là metadata, chữ ký, manifest của từng thư viện, không cần thiết cho việc chạy), em cho bỏ luôn (**discard**) để tránh xung đột.
- **case x => MergeStrategy.first**: với mọi file còn lại, nếu bị trùng thì em chọn giữ lại file gặp đầu tiên (**first**).

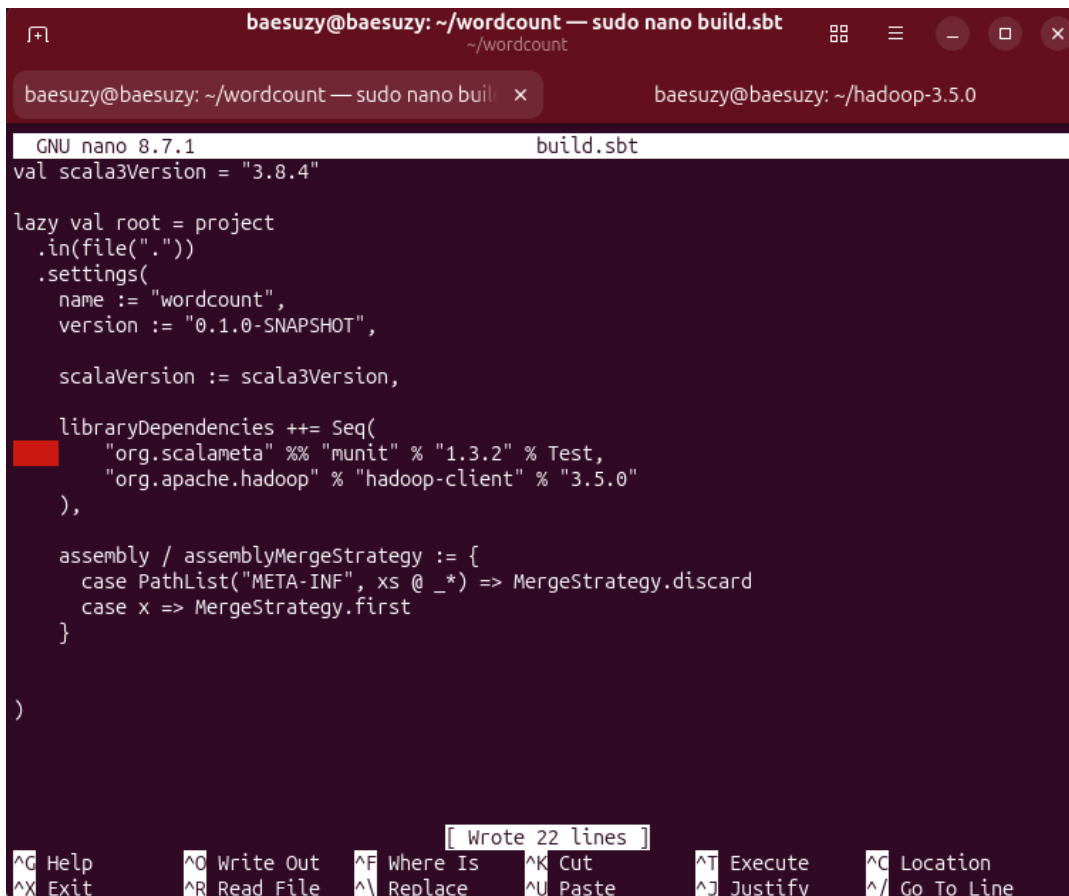
Nhờ khai báo chiến lược merge này, **sbt-assembly** mới gộp được toàn bộ dependencies vào một file JAR duy nhất mà không bị dừng vì lỗi conflict.

3. Build fat JAR:

```
1 sbt clean assembly
```

Kết quả như Hình 60.

4. Chạy lại thử với fat JAR:



```

baesuzy@baesuzy: ~/wordcount — sudo nano build.sbt
GNU nano 8.7.1 build.sbt
val scala3Version = "3.8.4"

lazy val root = project
  .in(file("."))
  .settings(
    name := "wordcount",
    version := "0.1.0-SNAPSHOT",

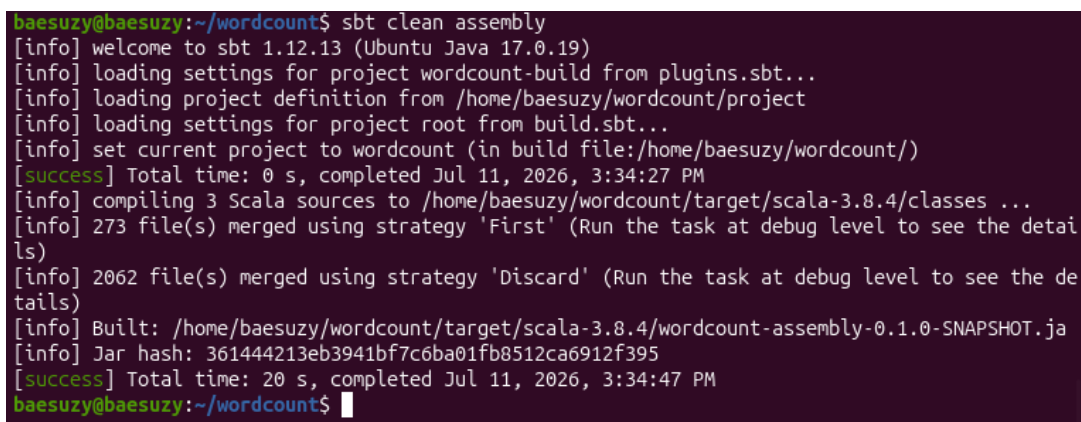
    scalaVersion := scala3Version,

    libraryDependencies ++= Seq(
      "org.scalameta" %% "munit" % "1.3.2" % Test,
      "org.apache.hadoop" % "hadoop-client" % "3.5.0"
    ),

    assembly / assemblyMergeStrategy := {
      case PathList("META-INF", xs @ _*) => MergeStrategy.discard
      case x => MergeStrategy.first
    }
  )
)

[ Wrote 22 lines ]
^C Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace    ^U Paste     ^J Justify   ^_ Go To Line
  
```

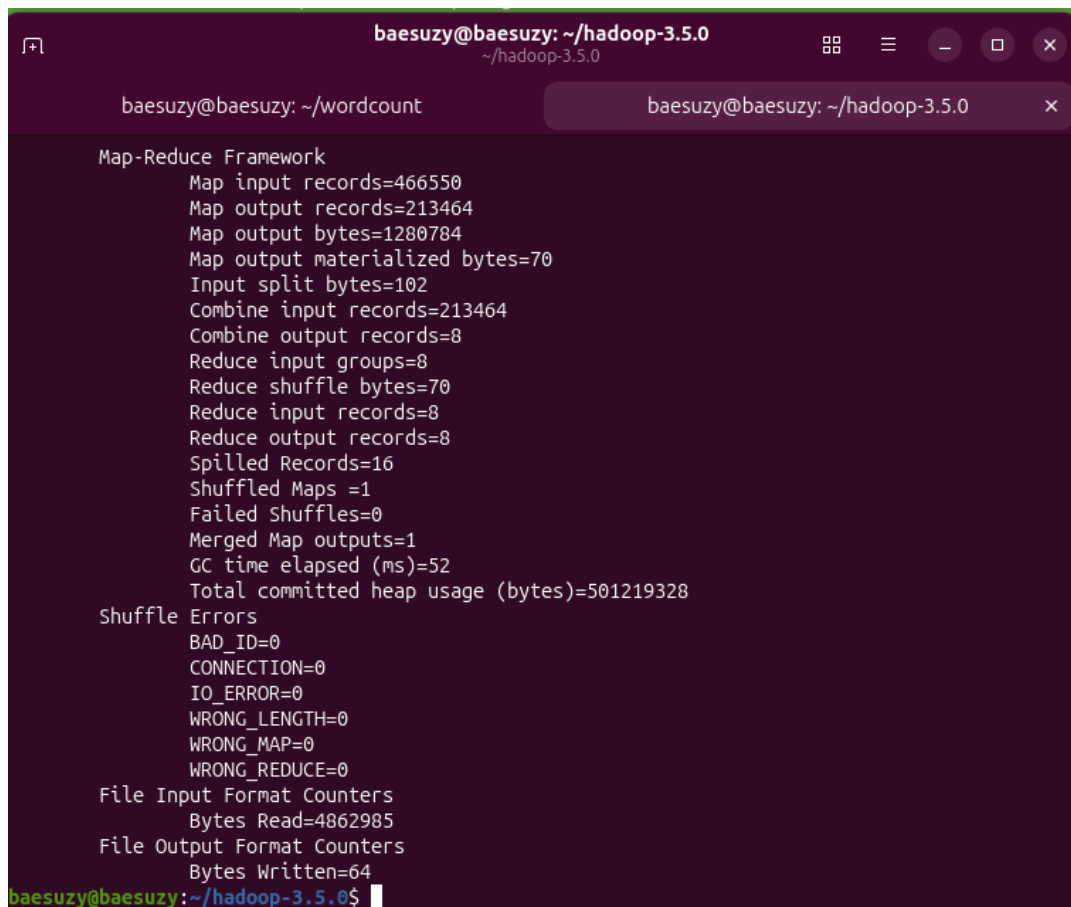
Hình 59: Khai báo assemblyMergeStrategy trong build.sbt.



```

baesuzy@baesuzy:~/wordcount$ sbt clean assembly
[info] welcome to sbt 1.12.13 (Ubuntu Java 17.0.19)
[info] loading settings for project wordcount-build from plugins.sbt...
[info] loading project definition from /home/baesuzy/wordcount/project
[info] loading settings for project root from build.sbt...
[info] set current project to wordcount (in build file:/home/baesuzy/wordcount/)
[success] Total time: 0 s, completed Jul 11, 2026, 3:34:27 PM
[info] compiling 3 Scala sources to /home/baesuzy/wordcount/target/scala-3.8.4/classes ...
[info] 273 file(s) merged using strategy 'First' (Run the task at debug level to see the details)
[info] 2062 file(s) merged using strategy 'Discard' (Run the task at debug level to see the details)
[info] Built: /home/baesuzy/wordcount/target/scala-3.8.4/wordcount-assembly-0.1.0-SNAPSHOT.jar
[info] Jar hash: 361444213eb3941bf7c6ba01fb8512ca6912f395
[success] Total time: 20 s, completed Jul 11, 2026, 3:34:47 PM
baesuzy@baesuzy:~/wordcount$
  
```

Hình 60: Build fat JAR bằng sbt clean assembly.



```
baesuzy@baesuzy: ~/hadoop-3.5.0
~/hadoop-3.5.0

baesuzy@baesuzy: ~/wordcount
baesuzy@baesuzy: ~/hadoop-3.5.0

Map-Reduce Framework
  Map input records=466550
  Map output records=213464
  Map output bytes=1280784
  Map output materialized bytes=70
  Input split bytes=102
  Combine input records=213464
  Combine output records=8
  Reduce input groups=8
  Reduce shuffle bytes=70
  Reduce input records=8
  Reduce output records=8
  Spilled Records=16
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=52
  Total committed heap usage (bytes)=501219328

Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0

File Input Format Counters
  Bytes Read=4862985

File Output Format Counters
  Bytes Written=64

baesuzy@baesuzy:~/hadoop-3.5.0$
```

Hình 61: Chạy WordCount với fat JAR thành công.

```
1 bin/hadoop jar ~/wordcount/target/scala-3.8.4/wordcount-assembly-0.1.0-SNAPSHOT.jar /input /output
```

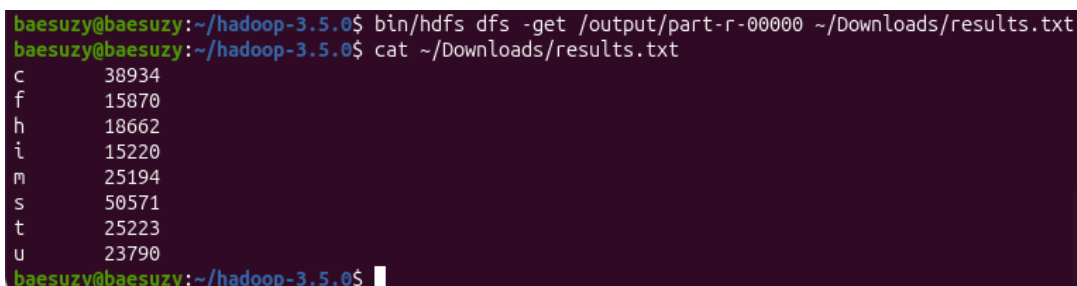
Kết quả như Hình 61.

Như vậy đã không còn thấy báo lỗi nữa. Giờ em sẽ check trên Web UI của Hadoop để xem kết quả output.

Truy cập vào <http://localhost:9870/explorer.html#/output> thì thấy có 1 file `_SUCCESS` và 1 file `part-r-00000`. Theo yêu cầu của lab thì em cần export file `part-r-00000` về máy local và lưu dưới dạng txt (tsv-formatted). Để làm được điều này, em sẽ dùng lệnh `bin/hdfs dfs -get` để copy file từ HDFS về máy local. Cụ thể, em sẽ chạy lệnh:

```
1 bin/hdfs dfs -get /output/part-r-00000 ~/Downloads/results.txt
```

Thử in ra nội dung file `results.txt` bằng lệnh `cat ~/Downloads/results.txt` (Hình 62).



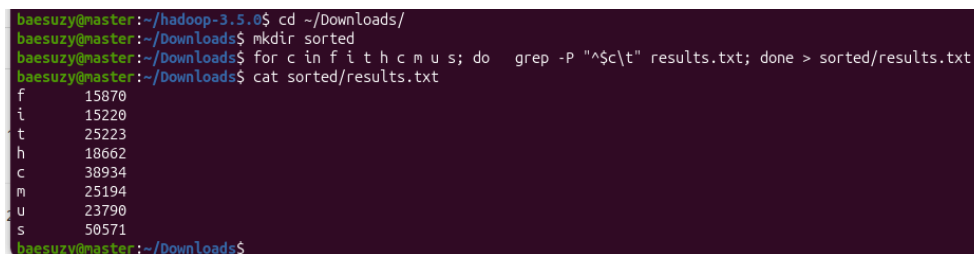
```
baesuzu@baesuzu:~/hadoop-3.5.0$ bin/hdfs dfs -get /output/part-r-00000 ~/Downloads/results.txt
baesuzu@baesuzu:~/hadoop-3.5.0$ cat ~/Downloads/results.txt
c      38934
f      15870
h      18662
i      15220
m      25194
s      50571
t      25223
u      23790
baesuzu@baesuzu:~/hadoop-3.5.0$
```

Hình 62: Nội dung file `results.txt` sau khi export.

Nhưng yêu cầu cần file đã sắp xếp theo thứ tự "fithcmus" nên em sẽ thực hiện thêm một thao tác nhỏ đơn giản bằng lệnh này.

```
1 mkdir sorted
2 for c in f i t h c m u s; do grep -P "^$c\t" results.txt; done > sorted/results.txt
3 cat sorted/results.txt
```

Đoạn lệnh trên sẽ cho ra kết quả như Hình 63.



```
baesuzu@master:~/hadoop-3.5.0$ cd ~/Downloads/
baesuzu@master:~/Downloads$ mkdir sorted
baesuzu@master:~/Downloads$ for c in f i t h c m u s; do grep -P "^$c\t" results.txt; done > sorted/results.txt
baesuzu@master:~/Downloads$ cat sorted/results.txt
f      15870
i      15220
t      25223
h      18662
c      38934
m      25194
u      23790
s      50571
baesuzu@master:~/Downloads$
```

Hình 63: Sắp xếp lại thứ tự trong `results.txt`

Tài liệu

- [1] Apache Hadoop. Hadoop: Setting up a single node cluster (pseudo-distributed operation). https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/SingleCluster.html#Pseudo-Distributed_Operation.
- [2] Apache Hadoop Wiki. Hadoop java versions. <https://cwiki.apache.org/confluence/spaces/HADOOP/pages/100827883/Hadoop+Java+Versions>.
- [3] Apache Software Foundation. Apache hadoop common releases. <https://dlcdn.apache.org/hadoop/common/>.
- [4] Canonical. Download ubuntu desktop. <https://ubuntu.com/download/desktop>.
- [5] Márton Salomváry. hadoop-mapreduce-tutorial: build.sbt. <https://github.com/salomvary/hadoop-mapreduce-tutorial/blob/master/build.sbt>.
- [6] sbt. Installing sbt on linux. <https://www.scala-sbt.org/1.x/docs/Installing-sbt-on-Linux.html>.
- [7] Scala Documentation. Getting started with scala and sbt on the command line. <https://docs.scala-lang.org/getting-started/h-scala-and-sbt-on-the-command-line.html>.
- [8] VMware. Workstation and fusion — desktop hypervisor. <https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion>.